

CS100433

Texture Mapping

Junqiao Zhao 赵君峤

School of Computer Science and Technology

Tongji University

Appearance property

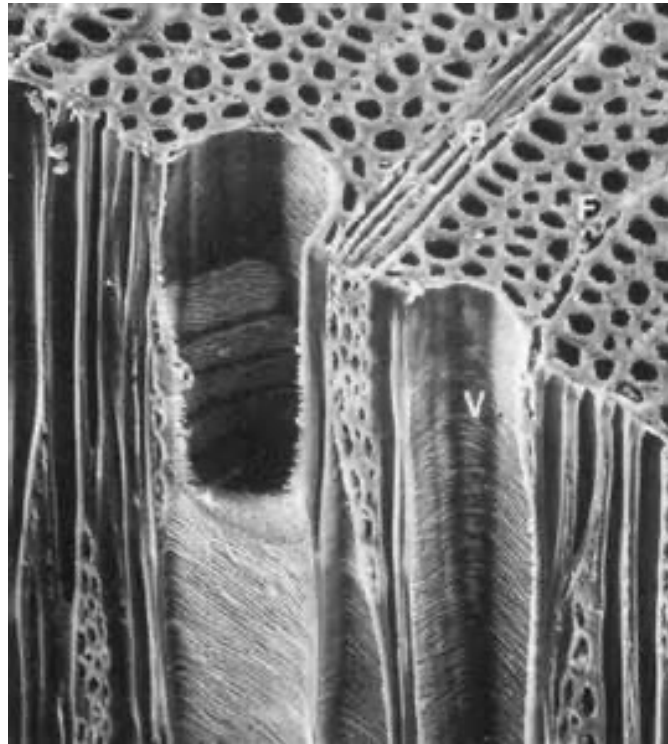
- Objects have geometry as well as appearance properties



[2013 Steve Marschner]

Appearance property

- In most scenarios, those properties are not modeled geometrically
- Why?



Appearance properties

- In many case such properties are demanded



[Tom Thorne, Edinburgh]

What is texture mapping?

- A technique of defining surface properties (especially shading parameters) in such a way that they vary as a function of position on the surface - Steve Marschner

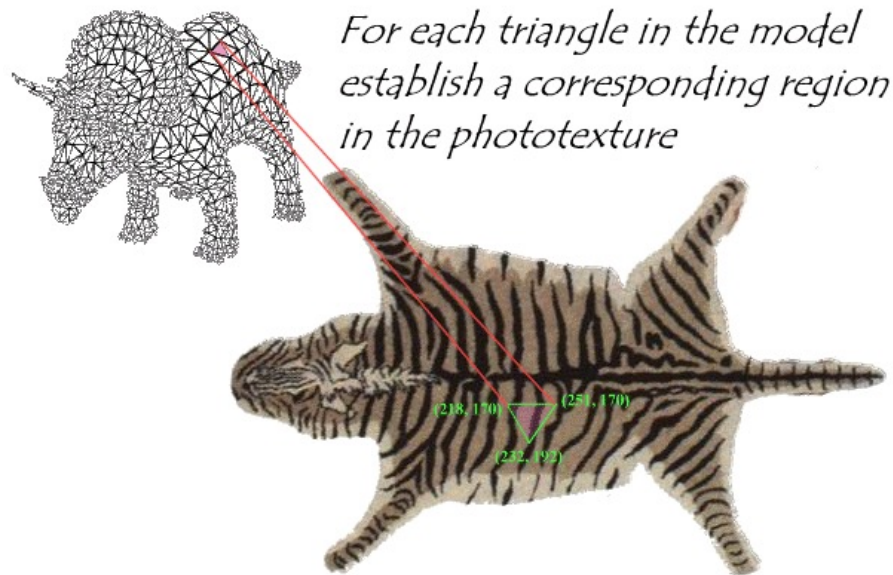


Pattern on ball

Wood grain on floor

What is texture mapping?

- An “image” is pasted onto the 3D model
- The “image” is called a **texture map**, the pixels of the image are referred as **texels** and referenced using **UV** coordinates

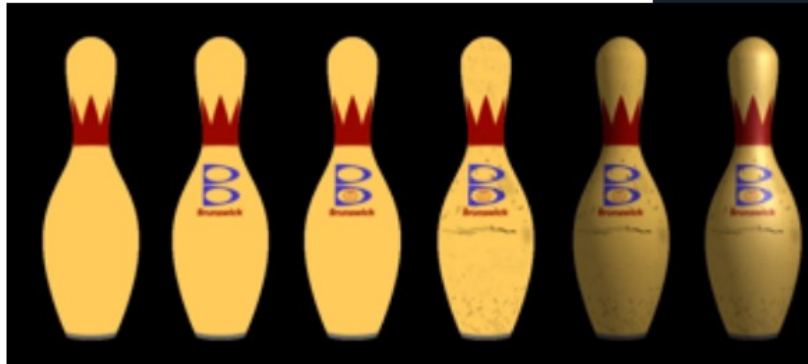


During rasterization interpolate the coordinate indices into the texture map

[Tom Thorne, Edinburgh]

Use of texture mapping

- Describe surface material properties

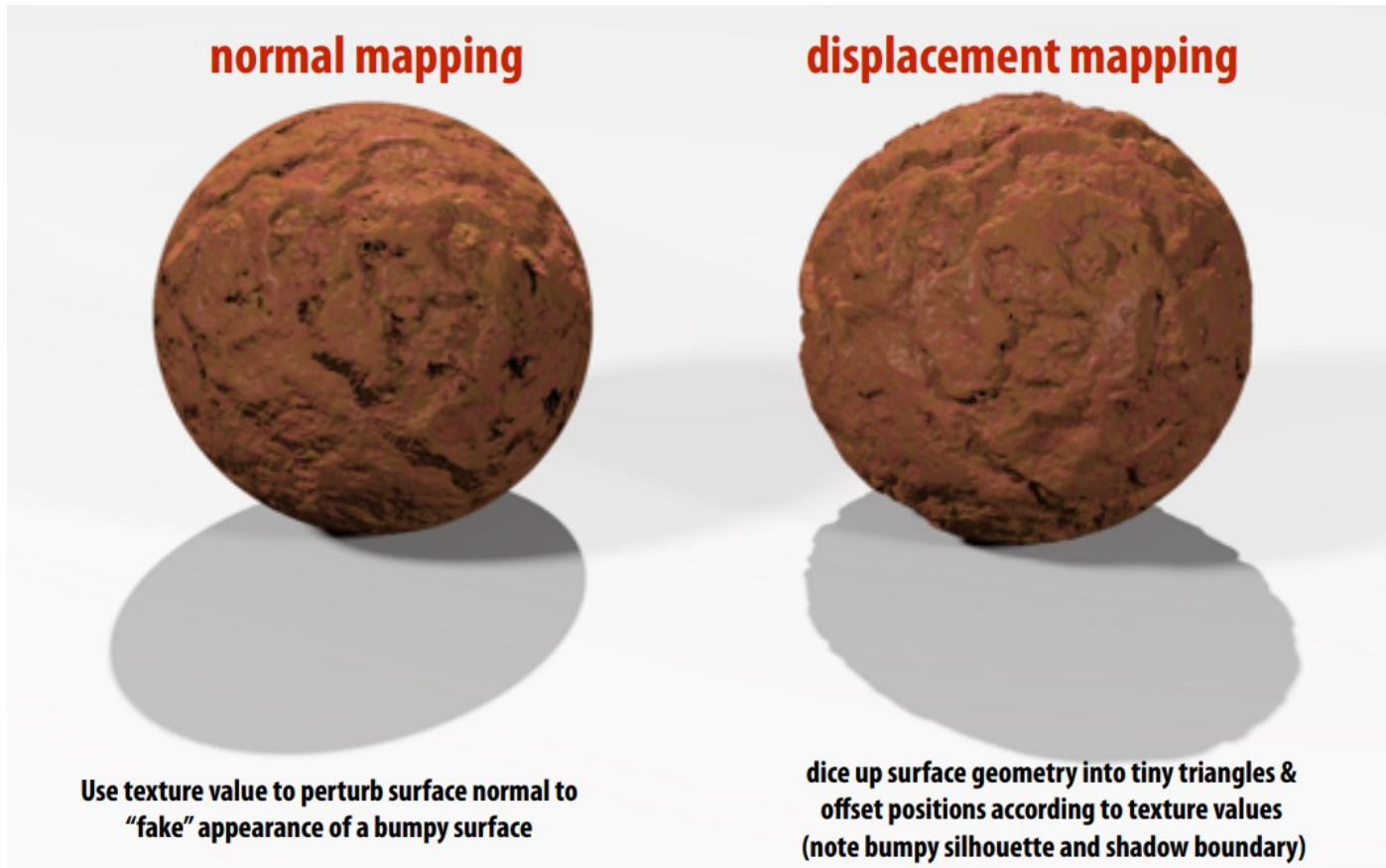


Multiple layers of texture maps for color, logos, scratches, etc.



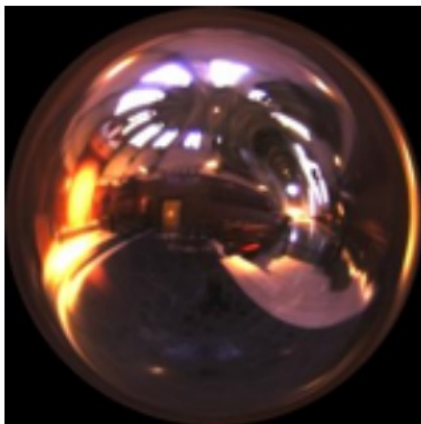
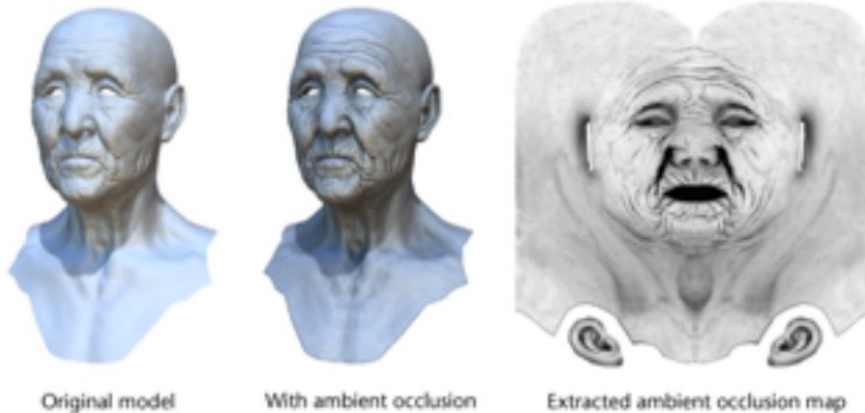
Use of texture mapping

- Normal and Displacement



Use of texture mapping

- Precomputed lighting and shadows



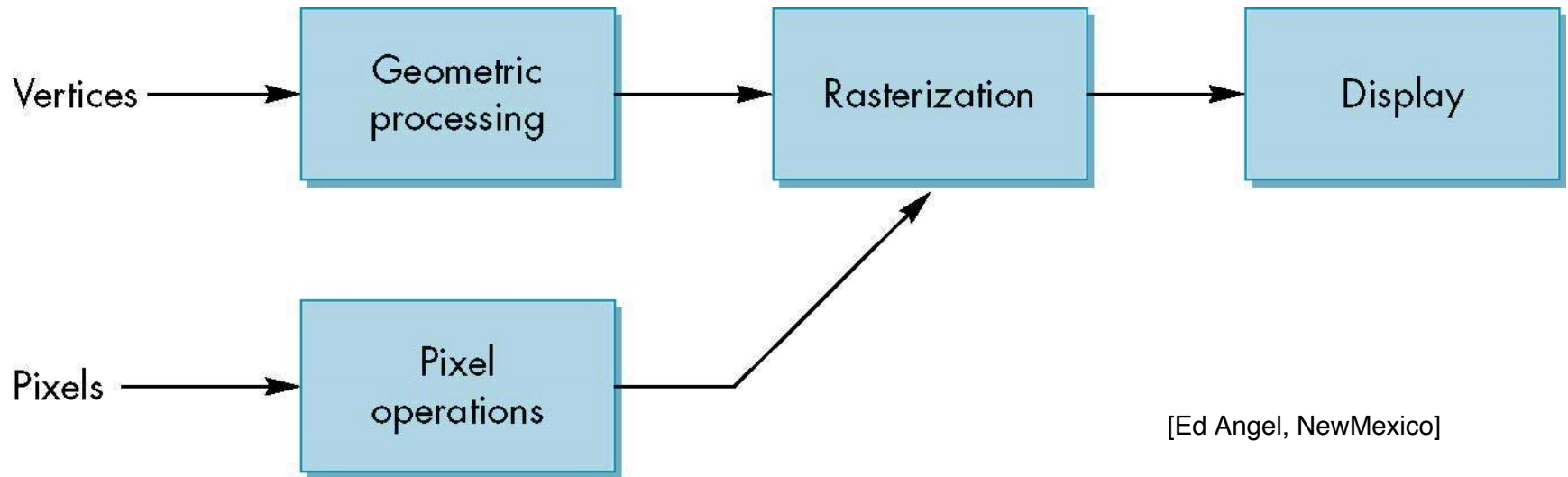
Grace Cathedral environment map



Environment map used in rendering

Where is texture mapping?

- Mapping techniques are implemented at the end of the rendering pipeline
 - Very efficient because polygons already passed the clipper



[Ed Angel, NewMexico]

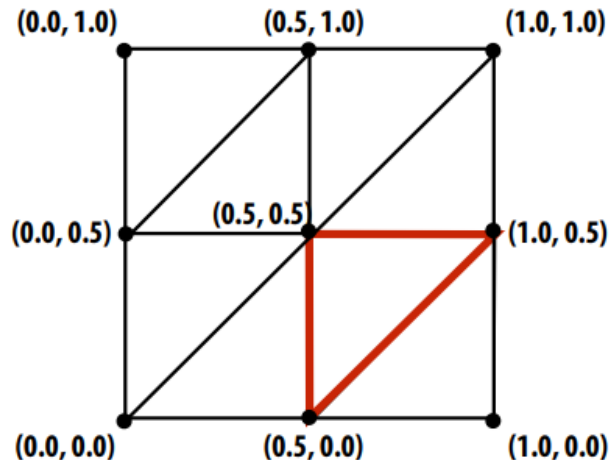
1D, 2D and 3D texture

- 1D/Linear texture
- 2D/Surface texture
- 3D/Volume texture

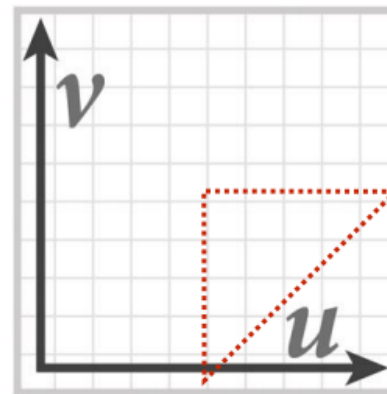
Surface texture mapping

- “Texture coordinates” define a mapping from surface coordinates to points in texture domain
- Often defined by linearly interpolating texture coordinates at triangle vertices

Suppose each cube face is split into eight triangles, with texture coordinates (u,v) at each vertex

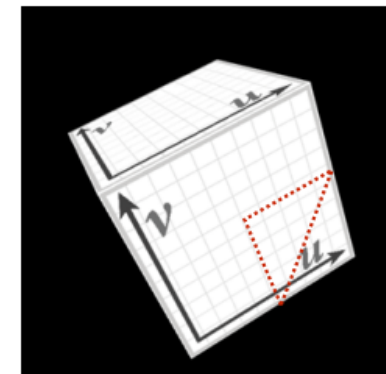


A texture on the $[0,1]^2$ domain can be specified by a 2048x2048 image



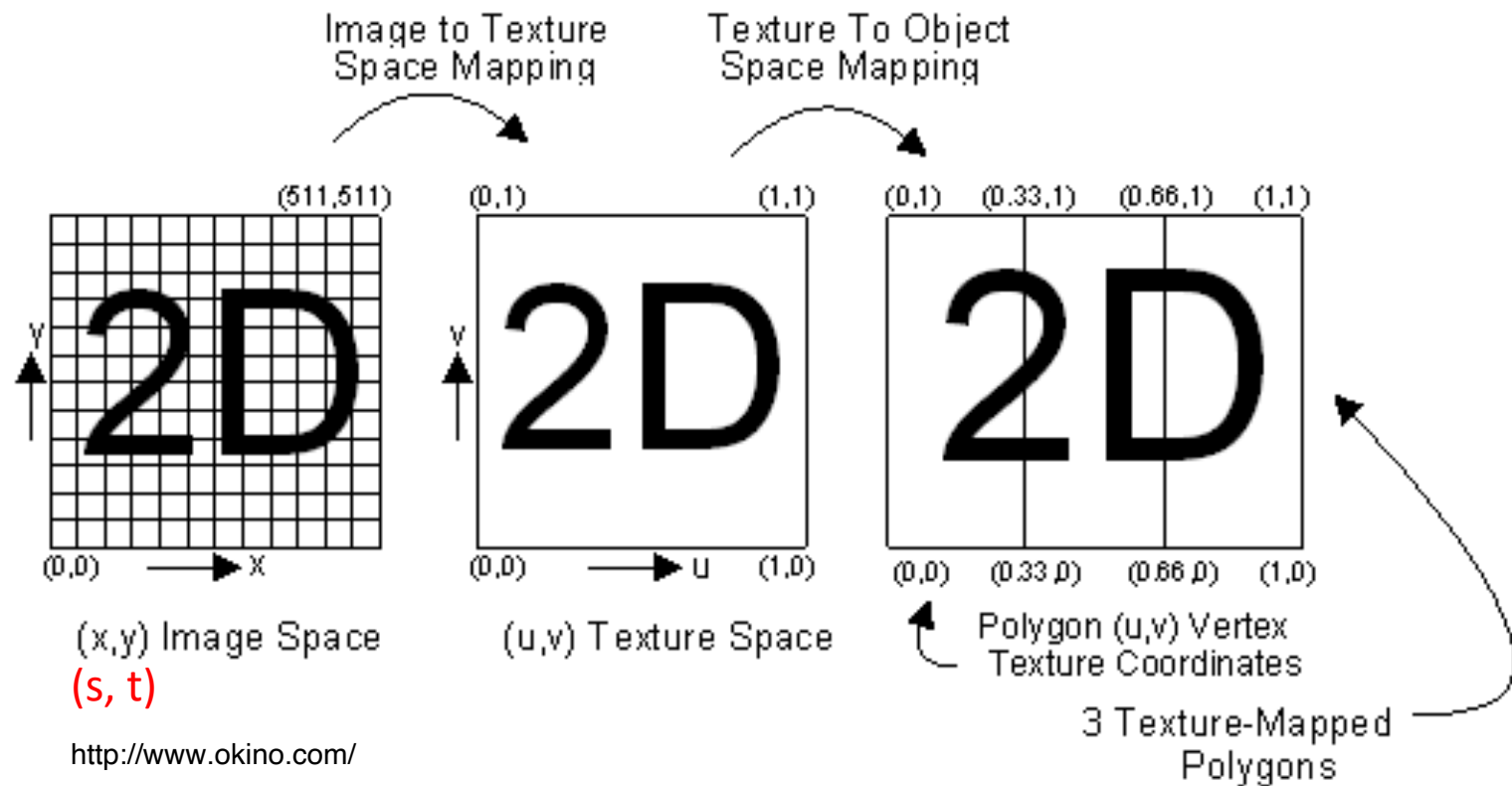
(location of highlighted triangle in texture space shown in red)

Linearly interpolating texture coordinates & “looking up” color in texture gives this image:



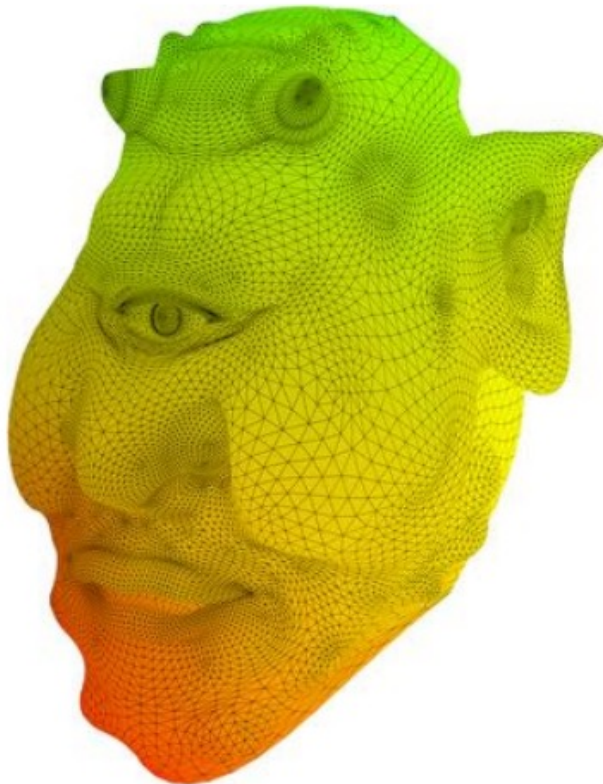
Surface texture mapping

- Texture coordinates (u, v) for each vertex

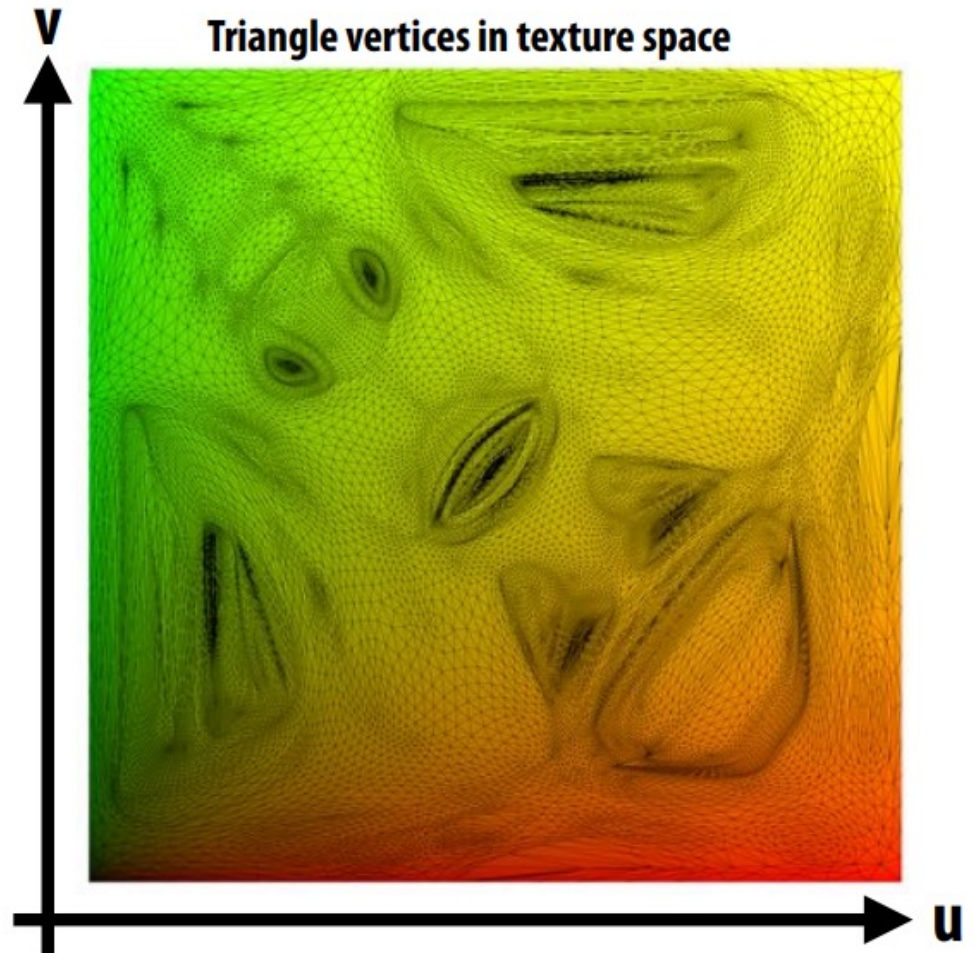


Visualization of texture coordinates

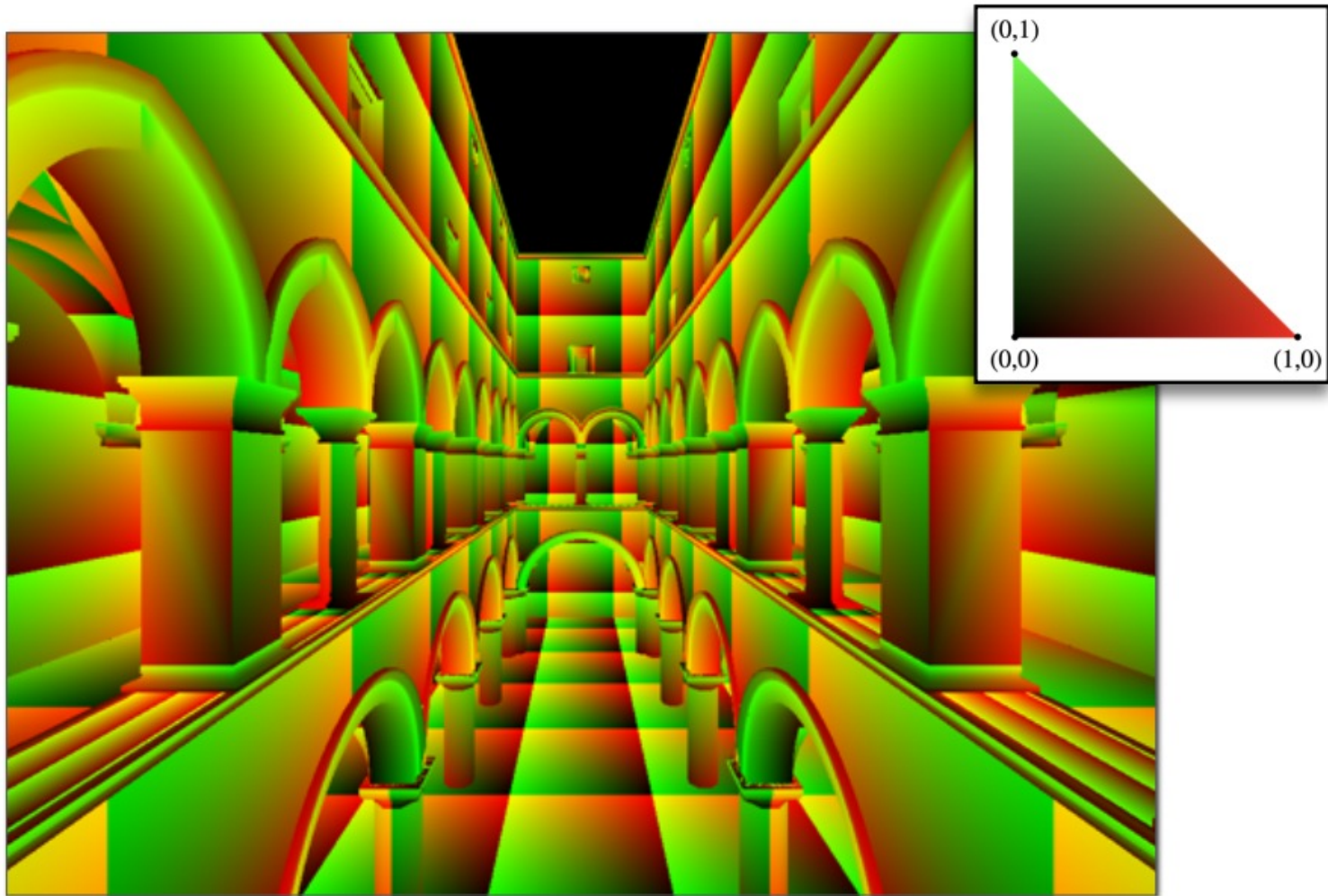
Visualization of texture coordinates



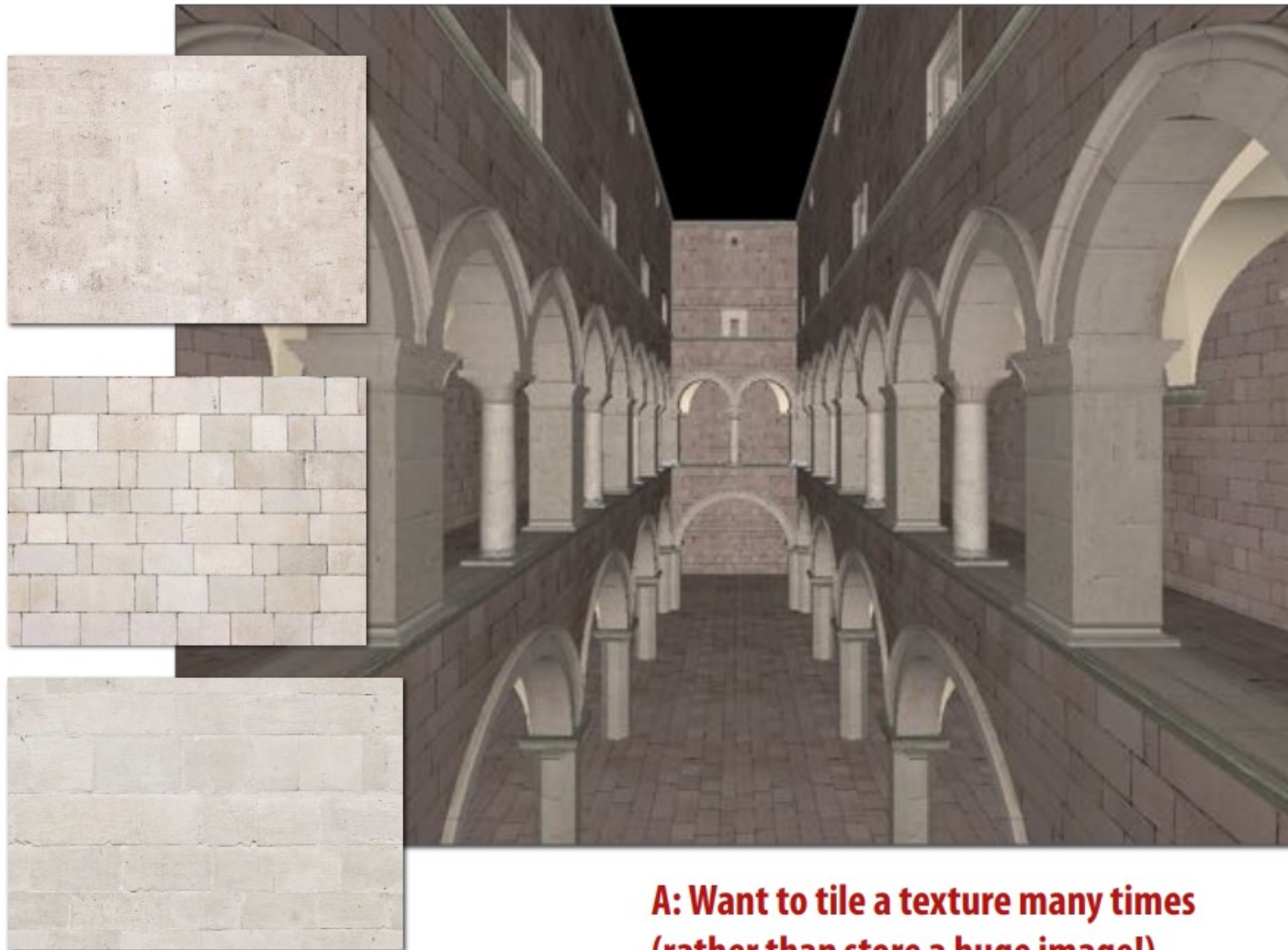
Triangle vertices in texture space



Visualization of texture coordinates



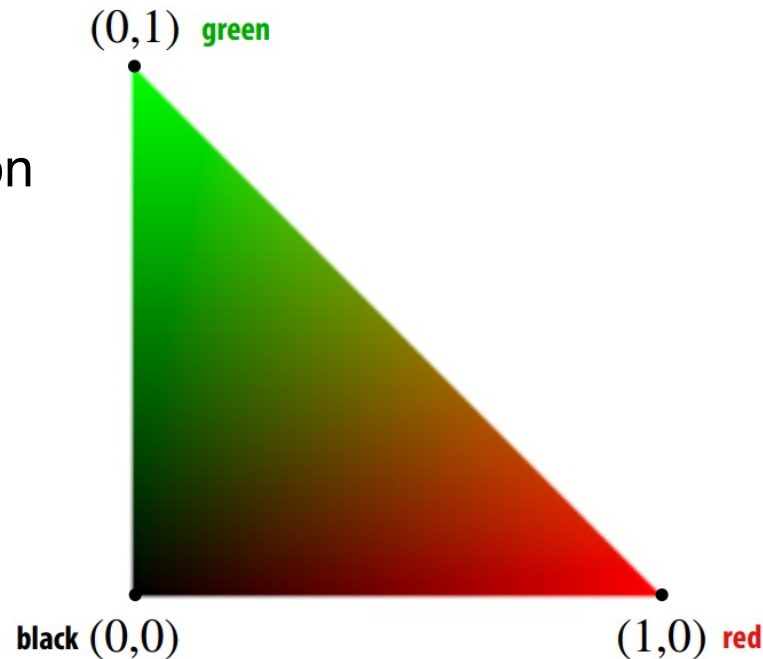
Visualization of texture coordinates



**A: Want to tile a texture many times
(rather than store a huge image!)**

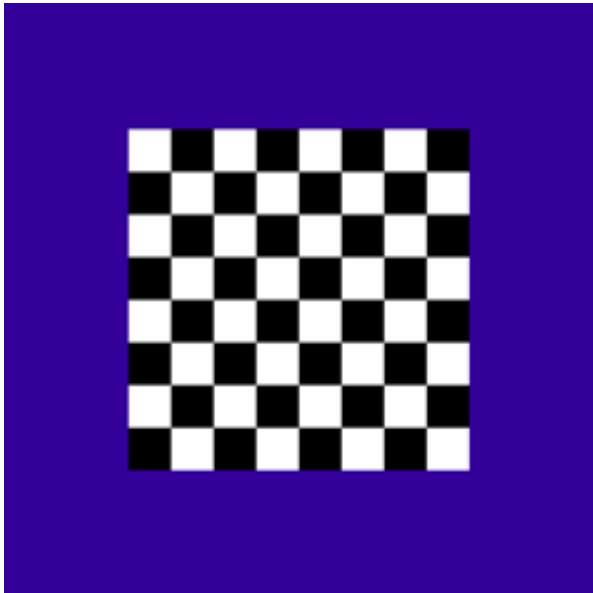
UV Interpolation

- Pixel coordinate to uv coordinates (2D to 2D)
- We can use barycentric coordinates or scanning conversion
- **However uv cannot be interpolated linearly in screen space**
 - Why?
 - Perspective distortion

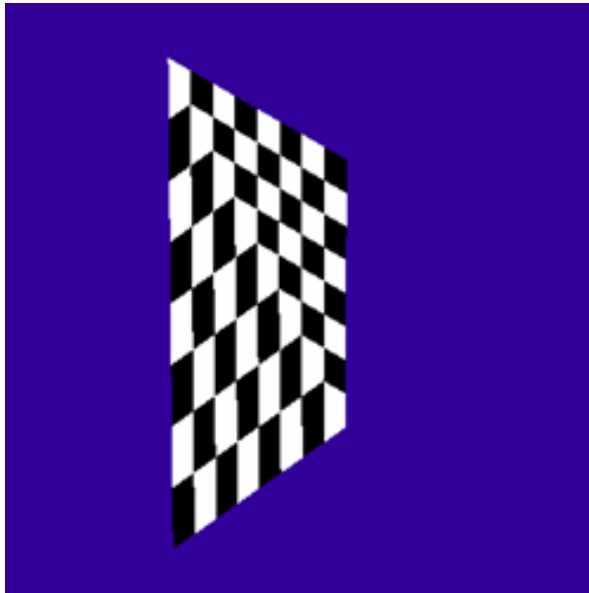


UV Interpolation

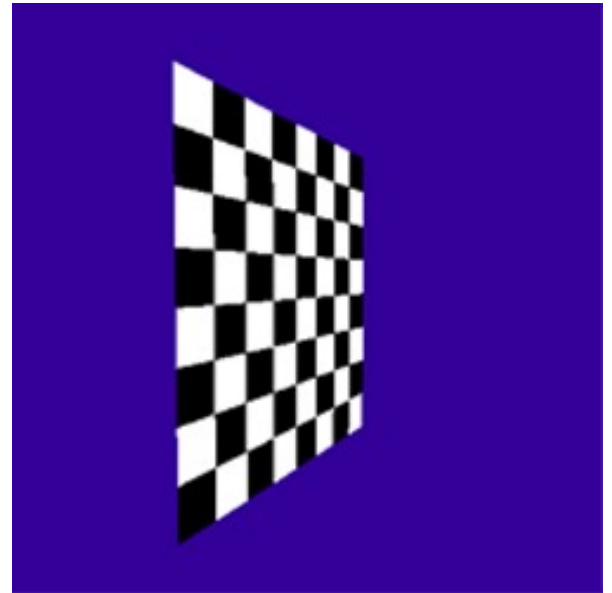
- Linear interpolation in screen space:



texture source



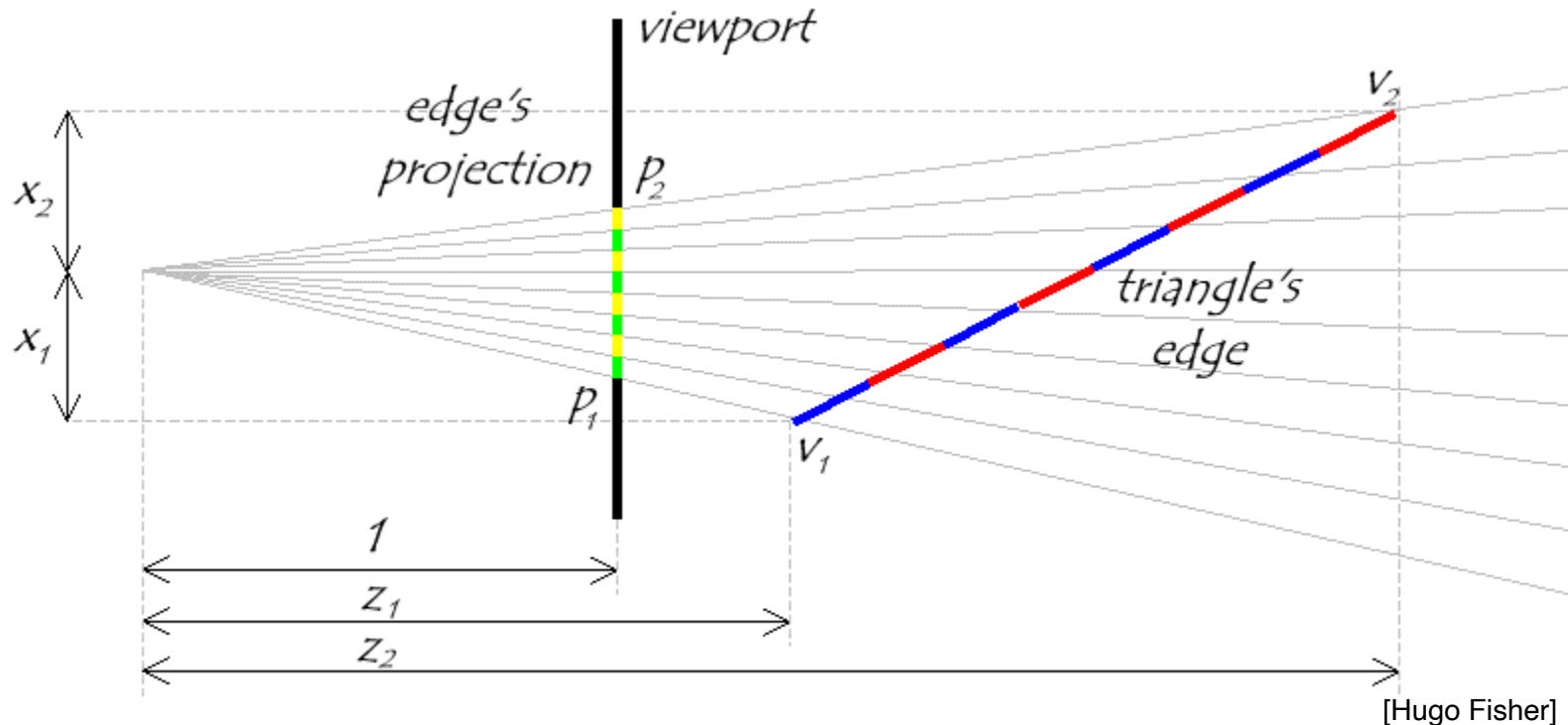
what we get



what we want

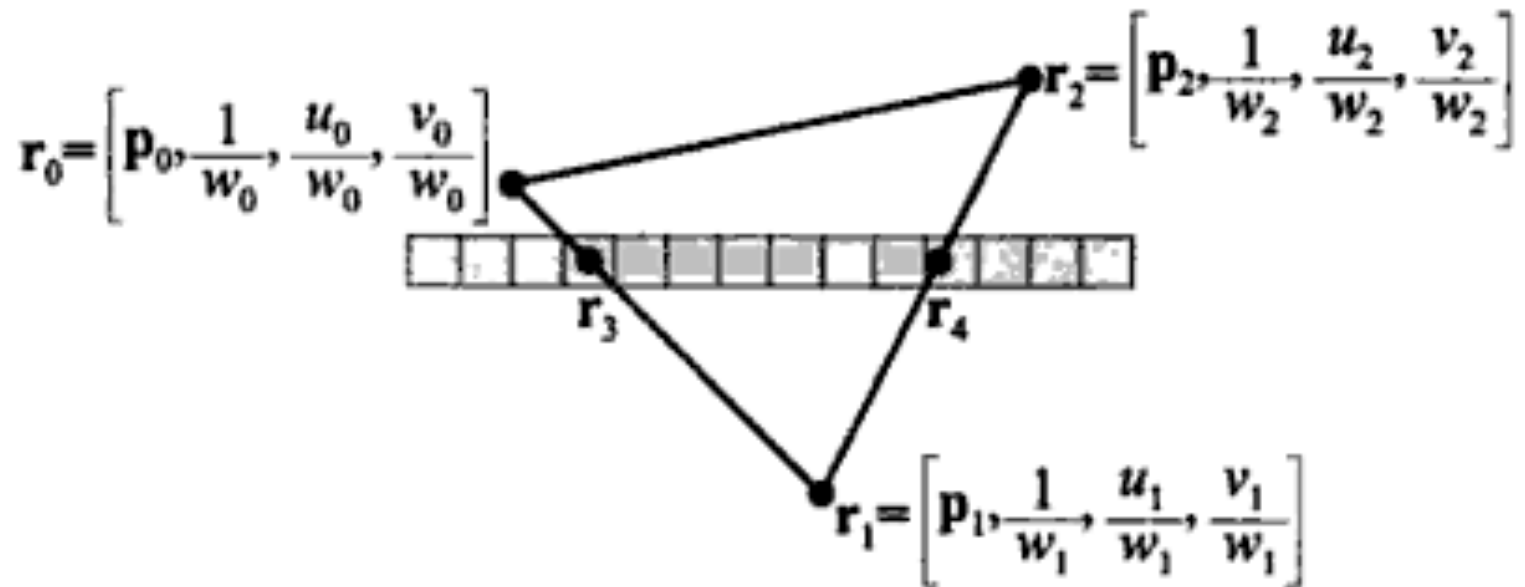
Perspective correction

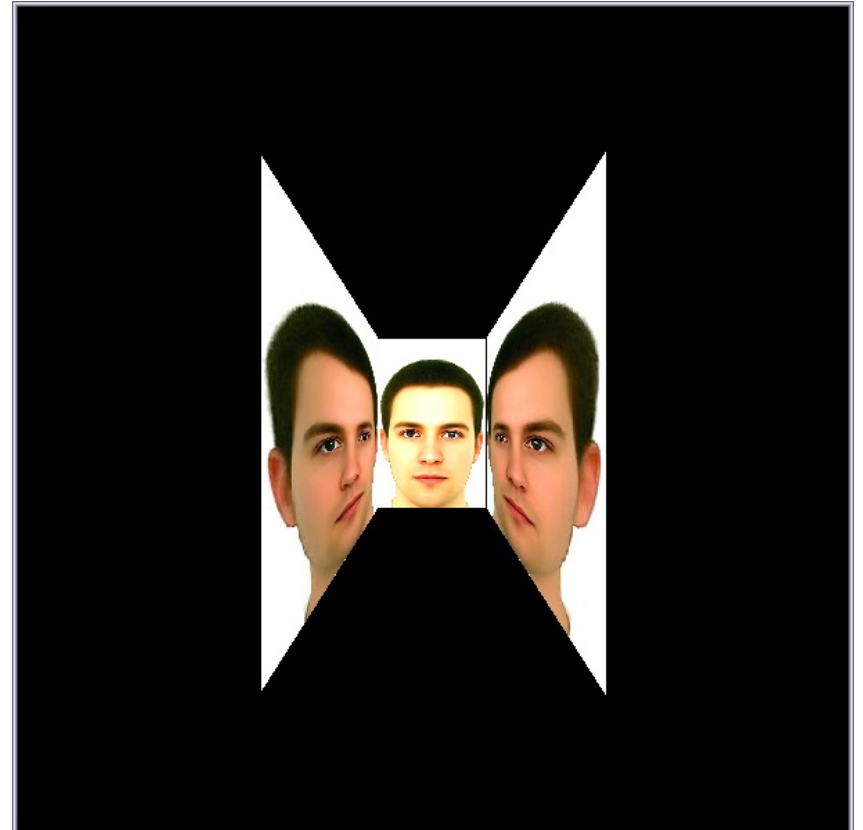
- Uniform steps on the image plane does not correspond to uniform steps along the edge



Perspective correction

- (u, v) should be divided by the depth
- using (u/w, v/w) in homogeneous coordinates

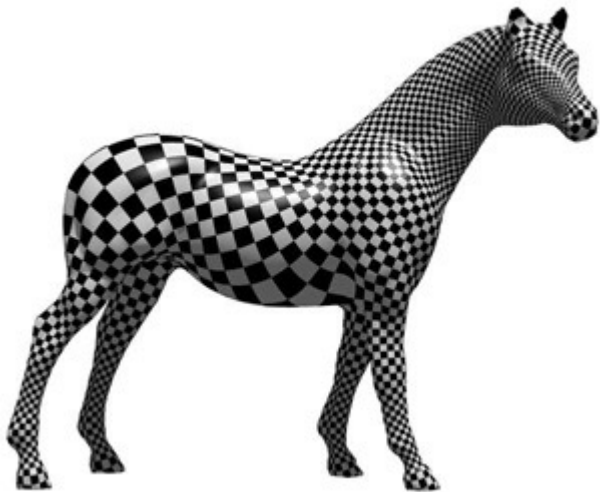




[Tom Thorne, Edinburgh]

Mesh parameterization (UVMapping)

- $u = u(x, y, z)$
- $v = v(x, y, z)$



[RiemannMapper]

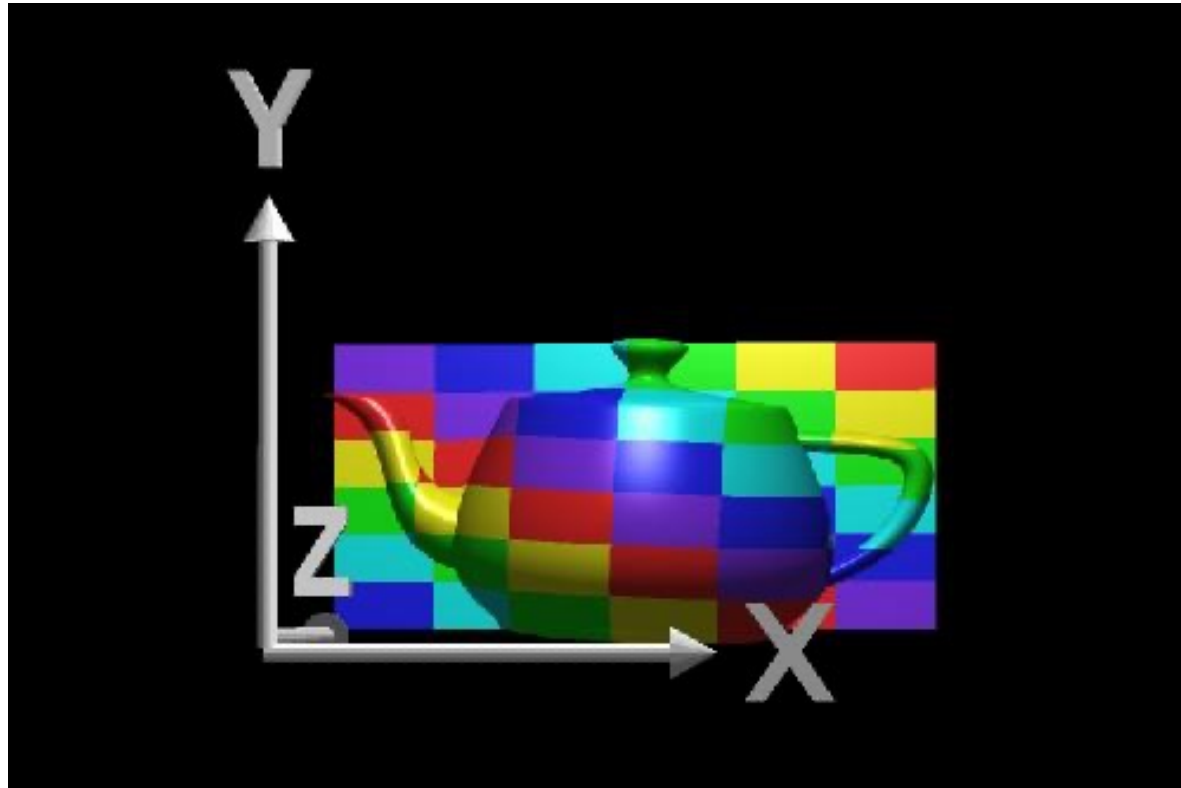


[CGAL]



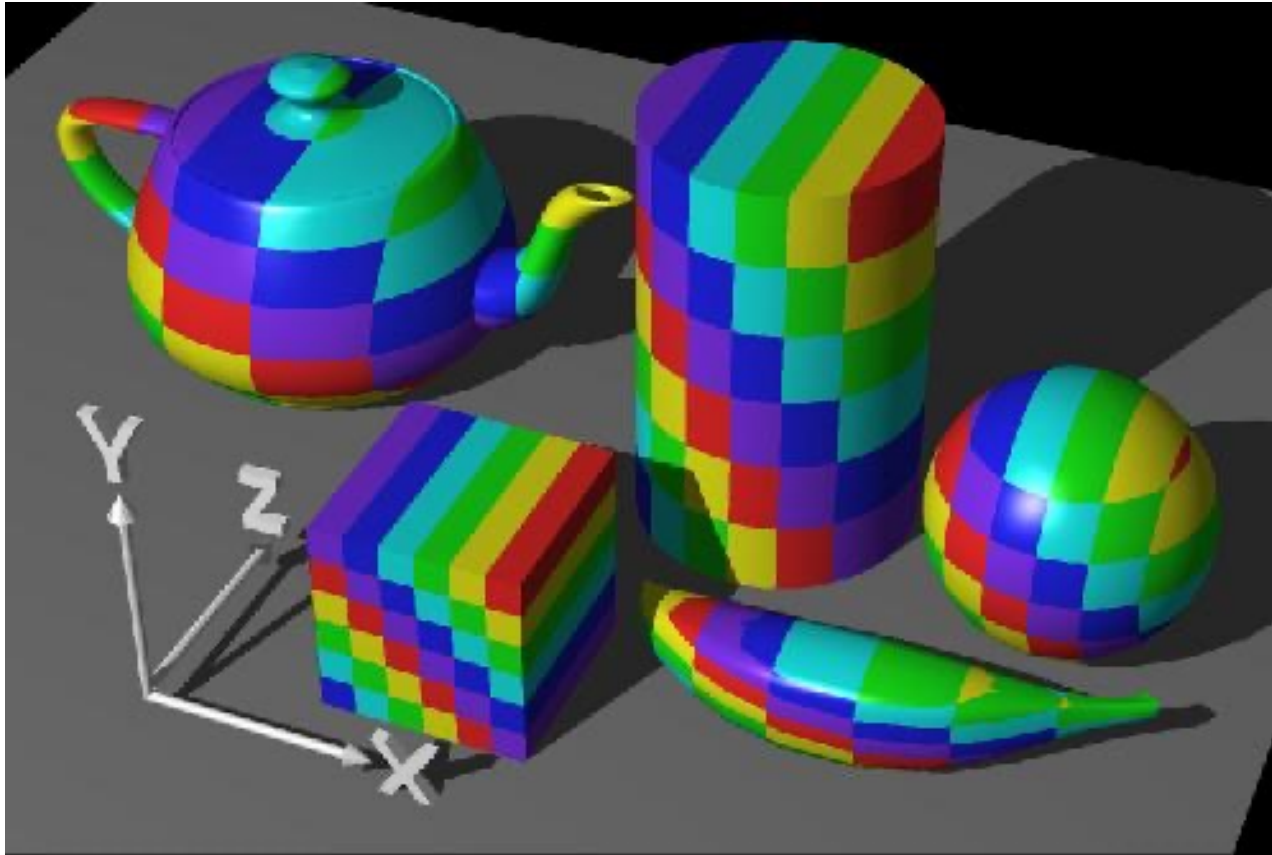
UVMapping - How to generate texture coordinates

- Where do we find the location in the texture map?



[Rosalee Wolfe / DePaul University]

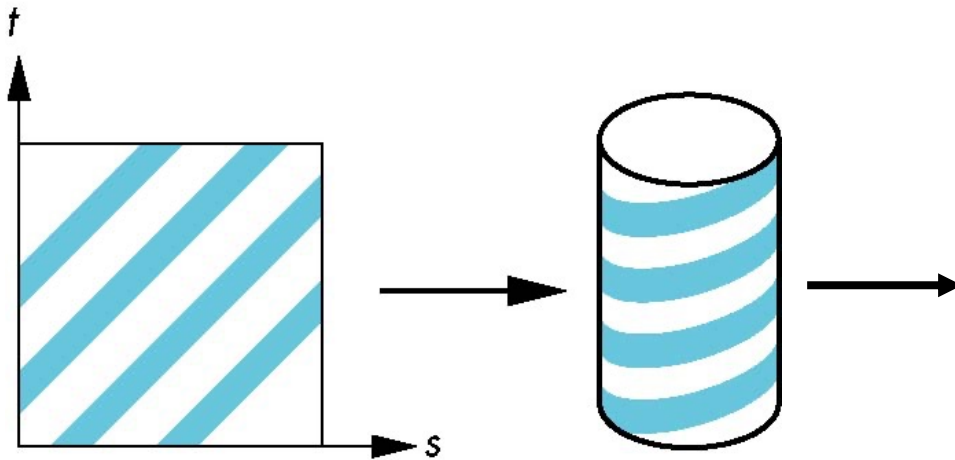
Orthogonal mapping



[Rosalee Wolfe / DePaul University]

Cylindrical mapping

- One solution is to first map the texture to an intermediate e.g. a cylinder or a sphere



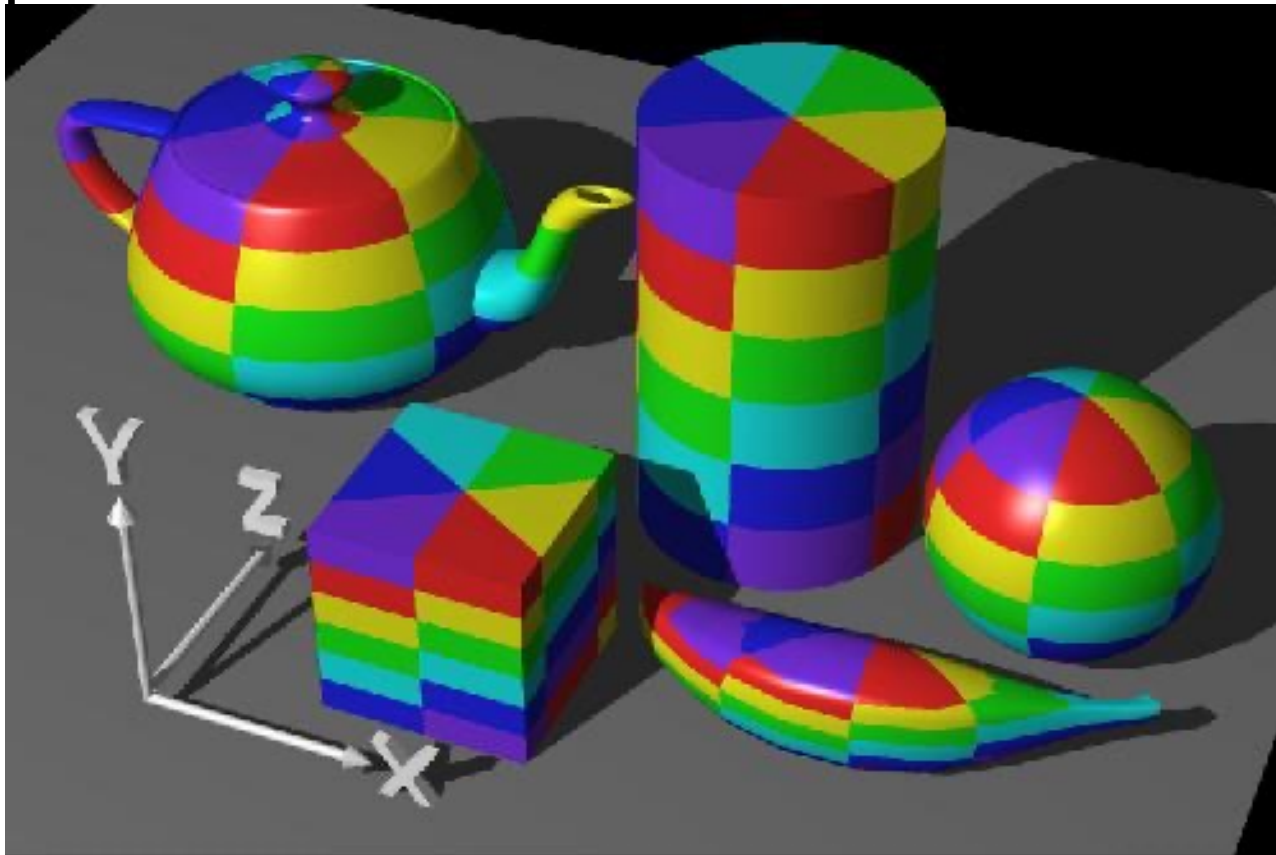
[Rosalee Wolfe / DePaul University]

Cylindrical mapping

- Cylinder parameterization
 - Using proper coordinate system (u, v)
 - $x = r \cos u$
 - $y = r \sin u$
 - $z = v$
 - Then find $u = u(s, t), v = v(s, t)$ which is linear
 - $s = 0 \rightarrow u = 0, s = 1 \rightarrow u = 2\pi$
 - $t = 0 \rightarrow v = 0, t = 1 \rightarrow v = h$
 - $u = 2\pi s, v = ht$
 - This maps u, v space to cylinder of radius r and height h in world coordinates

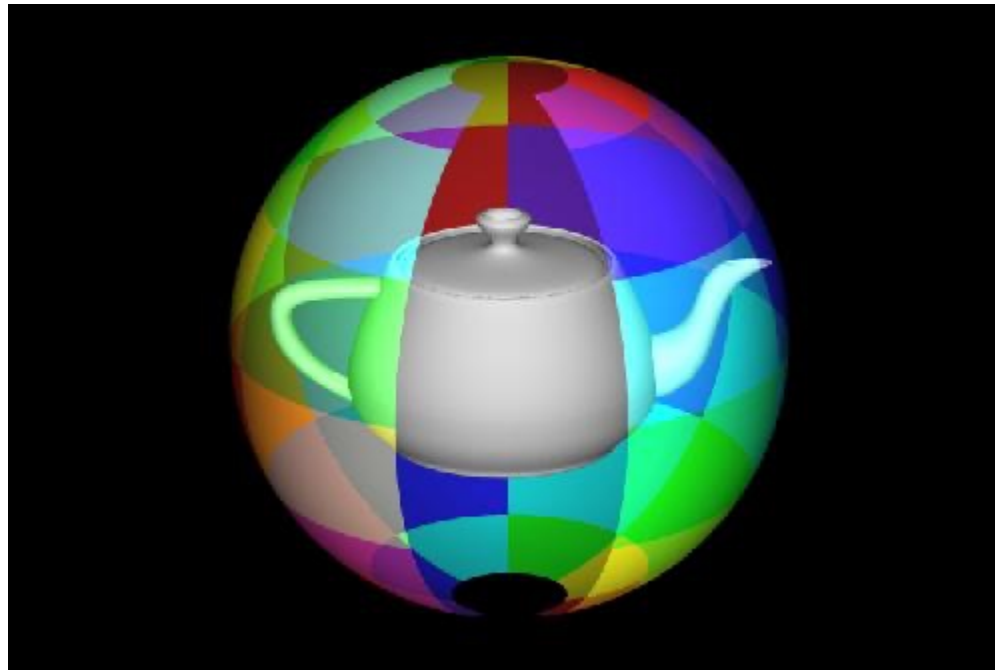
Cylindrical mapping

- Then the uv is mapped orthogonally to the shape



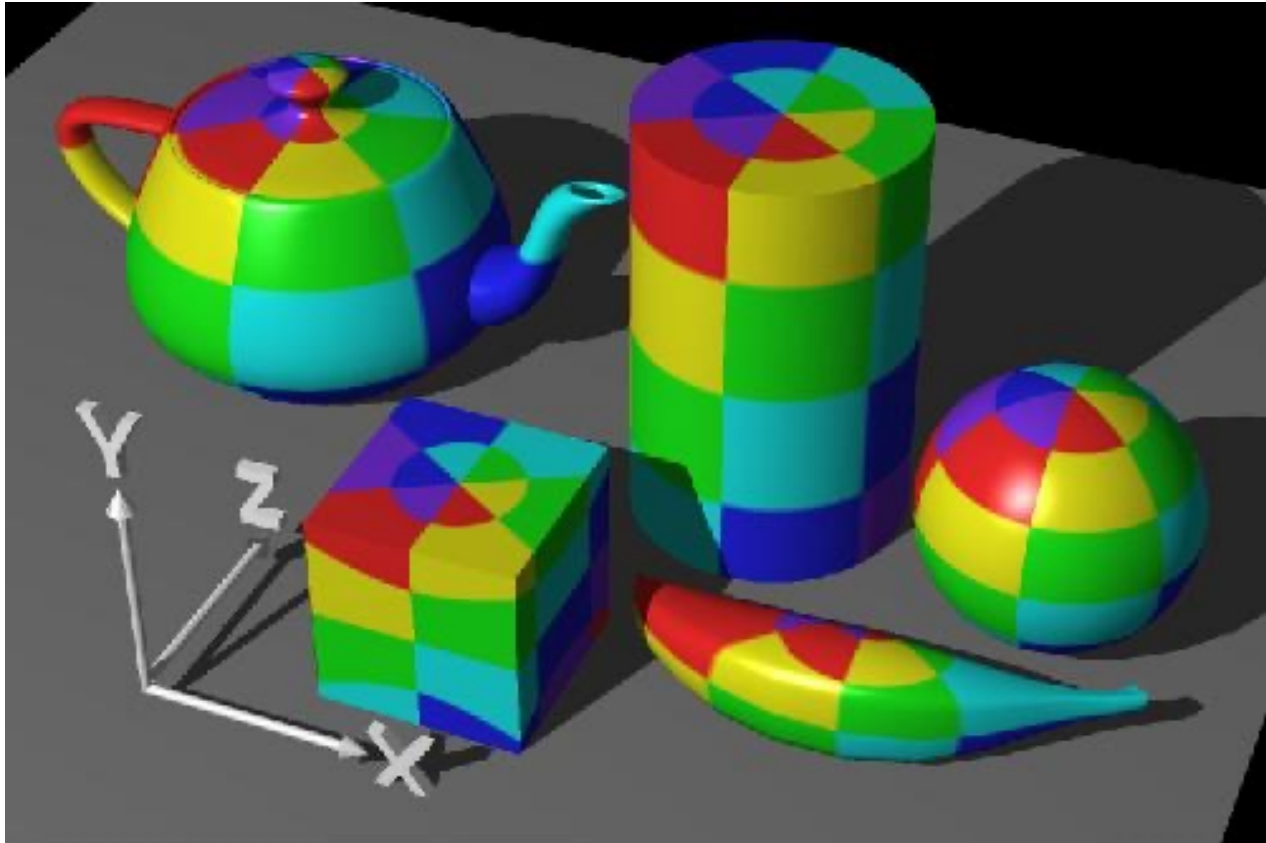
Spherical mapping

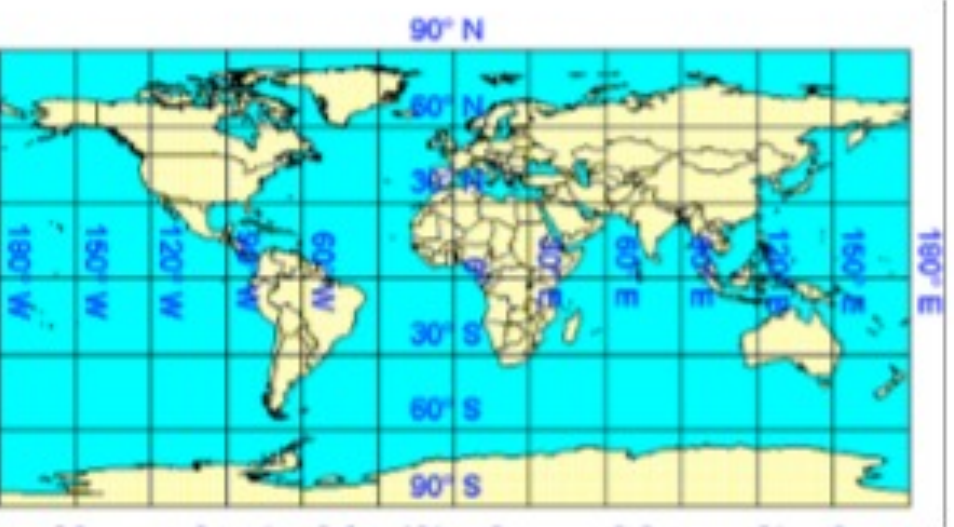
- We can further use a parameterized sphere
 - $x = r \cos u$
 - $y = r \sin u \cdot \cos v$
 - $z = r \sin u \cdot \sin v$



Spherical mapping

- Then the uv is mapped orthogonally to the shape





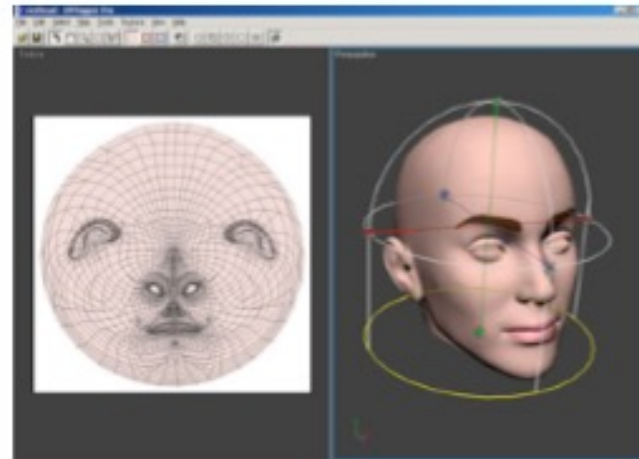
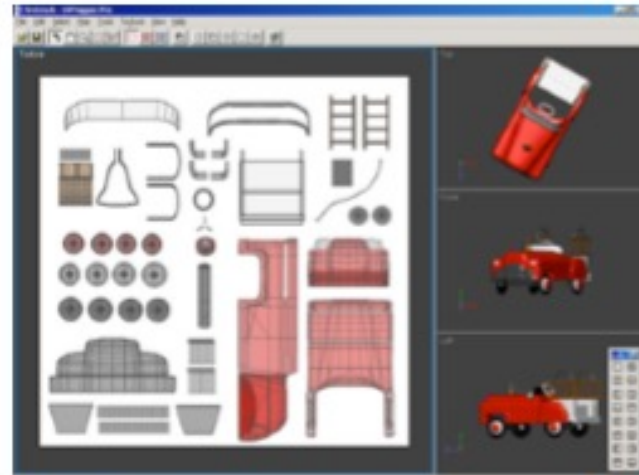
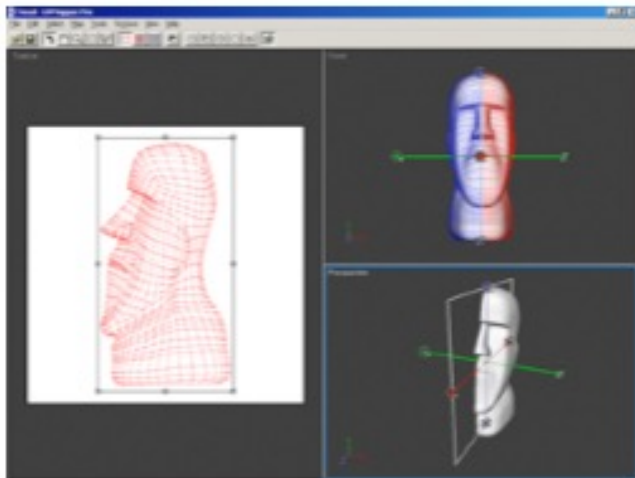
[map: Peter H. Dana]

UVMapping

- UV mapper in modeling software

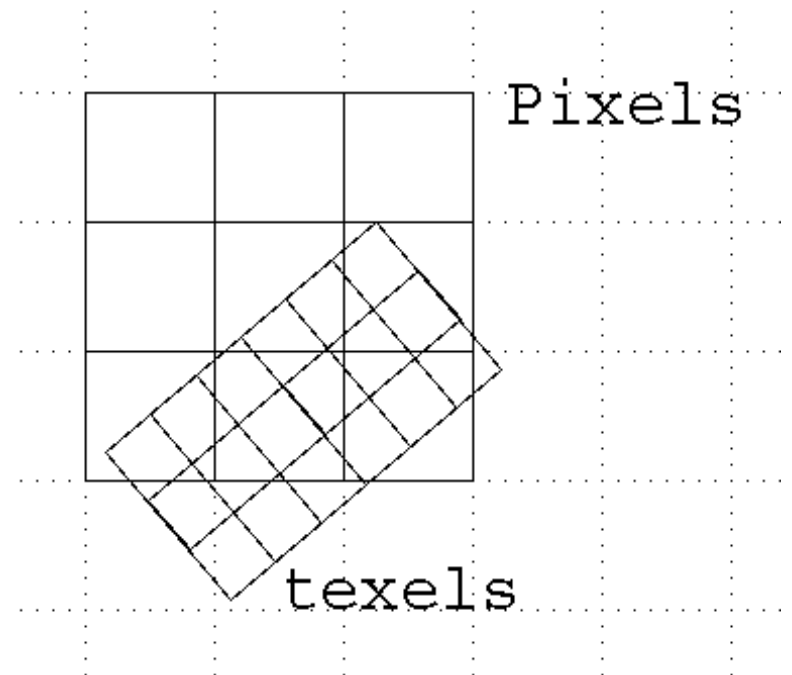
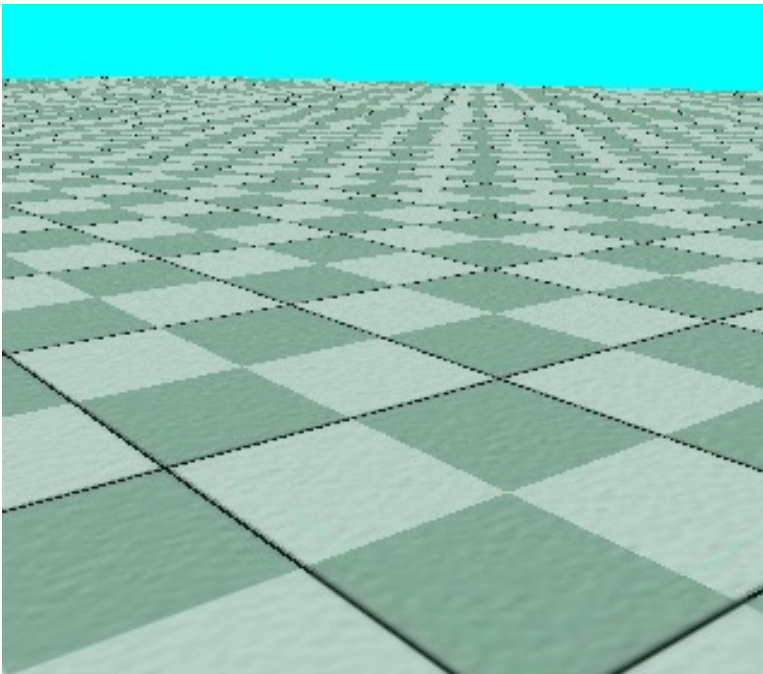
Example: UVMapper

<http://www.uvmapper.com>



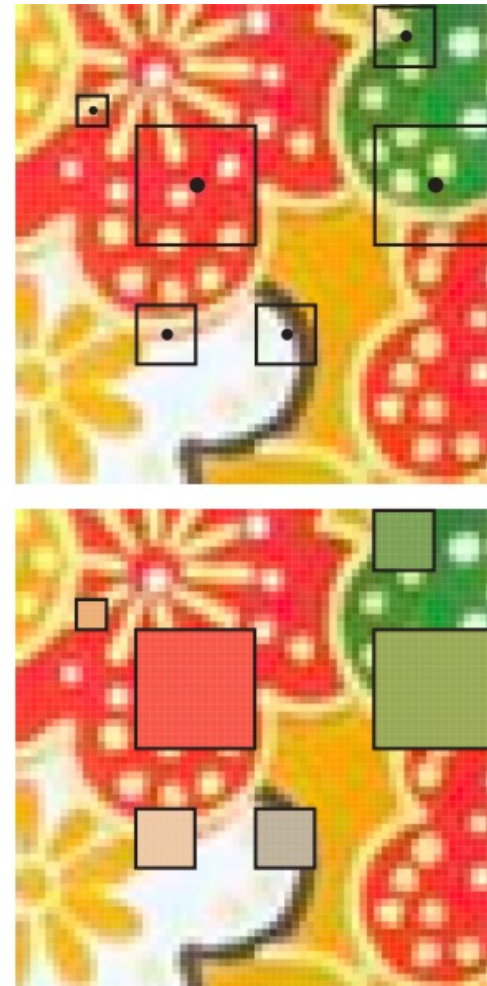
Aliasing of textures

- When several texels covering a pixel
- or the textured surface is magnificated too much (we see pixels!)



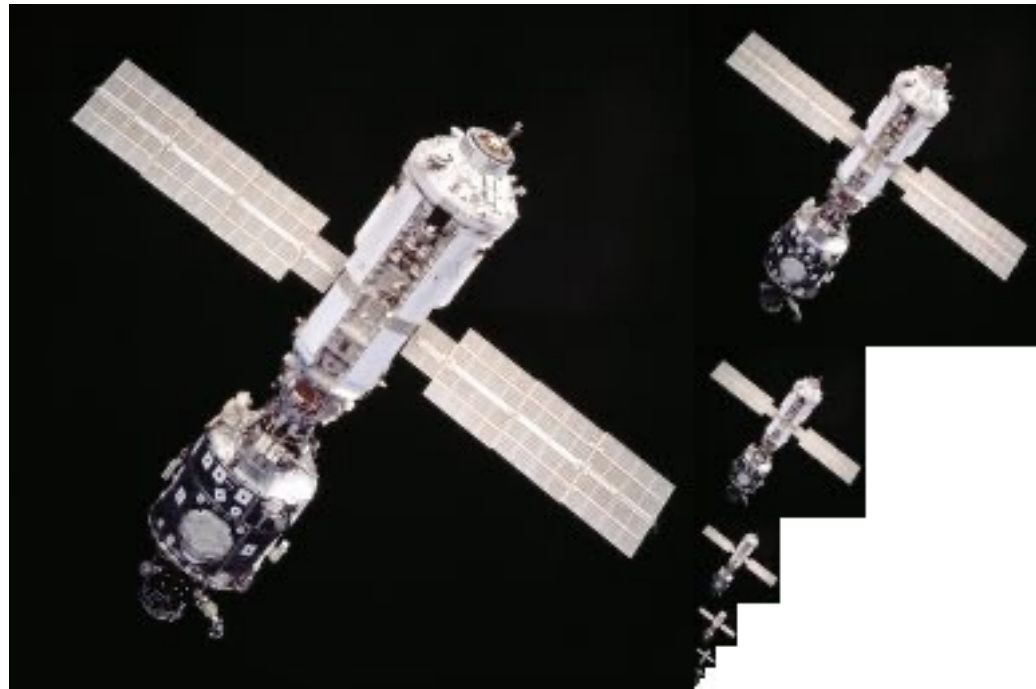
Texture prefiltering

- If we just grab the texture value at the center of the pixel, we get aliasing (get a “random” color that changes if the sample moves even very slightly)
- Ideally, would use the average texture value—but this is expensive to compute
- Instead, we can pre-compute the averages (once) and just look up these averages (many times) at run-time



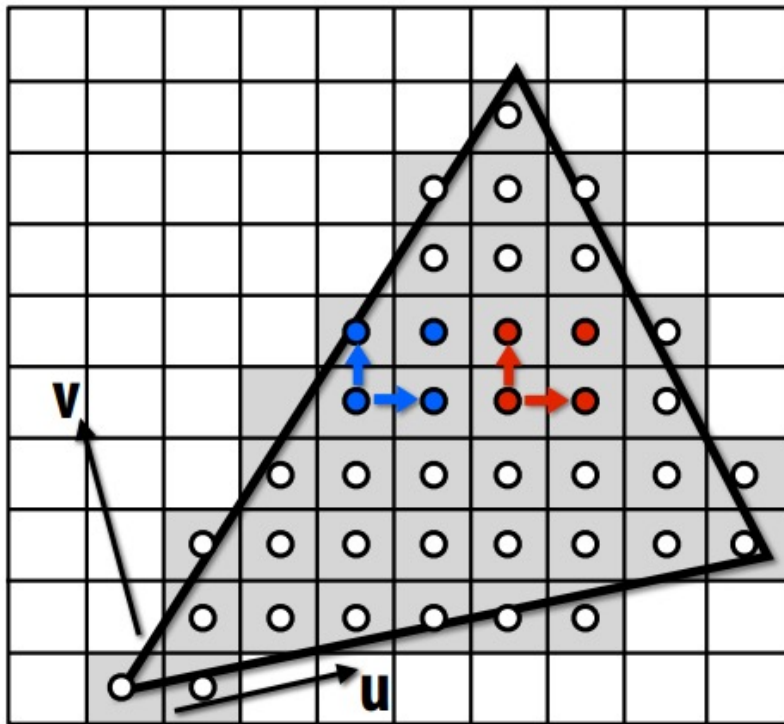
Mipmapping

- Use a texture of multiple resolutions
- Switch the resolution according to the number of texels in one pixel

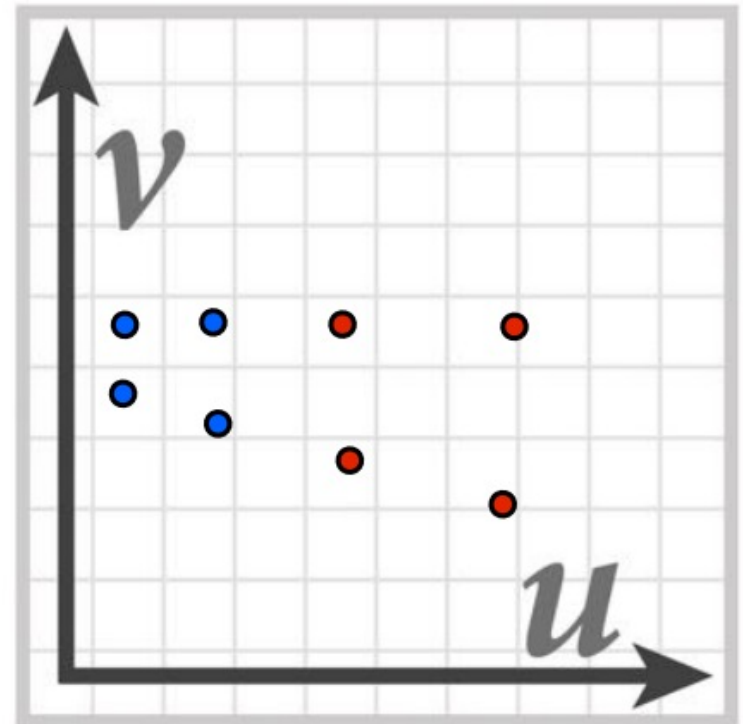


Computing Mipmapping Level

Even within a single triangle, may want to sample from different MIP map levels:



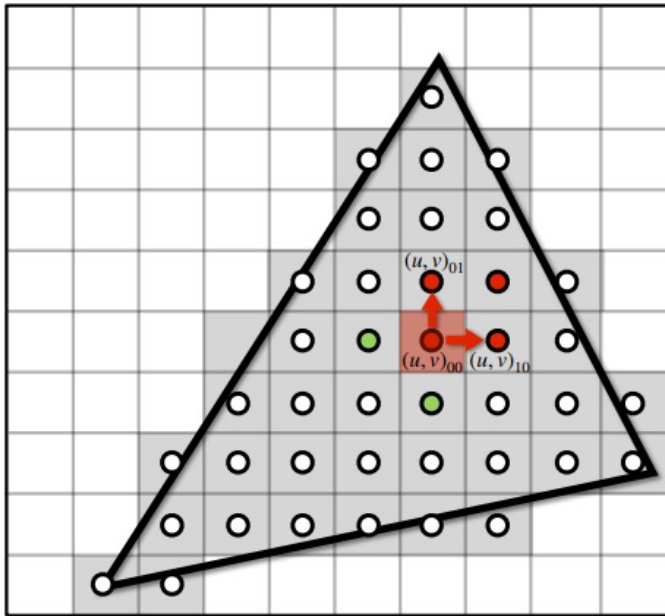
Screen space



Texture space

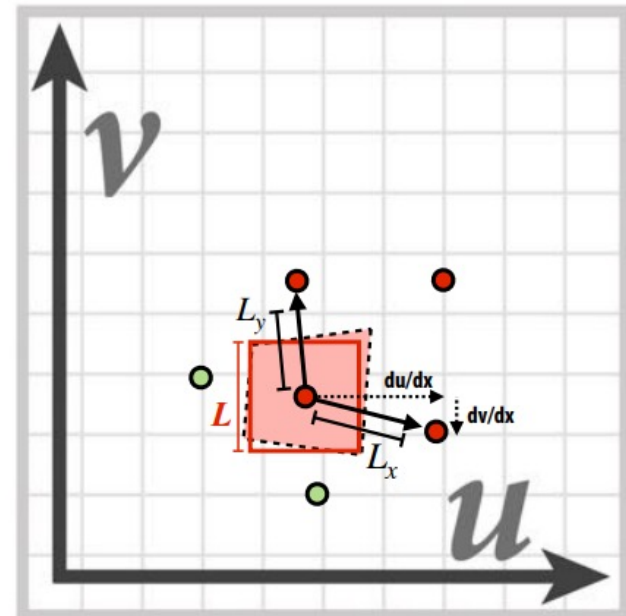
Computing Mipmapping Level

Compute differences between texture coordinate values at neighboring samples



$$\frac{du}{dx} = u_{10} - u_{00} \quad \frac{dv}{dx} = v_{10} - v_{00}$$

$$\frac{du}{dy} = u_{01} - u_{00} \quad \frac{dv}{dy} = v_{01} - v_{00}$$



$$L_x^2 = \left(\frac{du}{dx}\right)^2 + \left(\frac{dv}{dx}\right)^2 \quad L_y^2 = \left(\frac{du}{dy}\right)^2 + \left(\frac{dv}{dy}\right)^2$$

$$L = \sqrt{\max(L_x^2, L_y^2)}$$

mip-map level: $d = \log_2 L$





256x256

LOD0



128x128

LOD1



64x64

LOD2



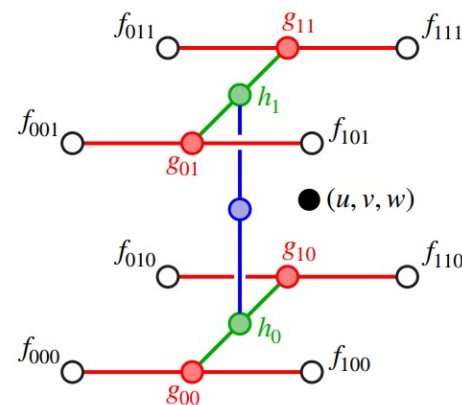
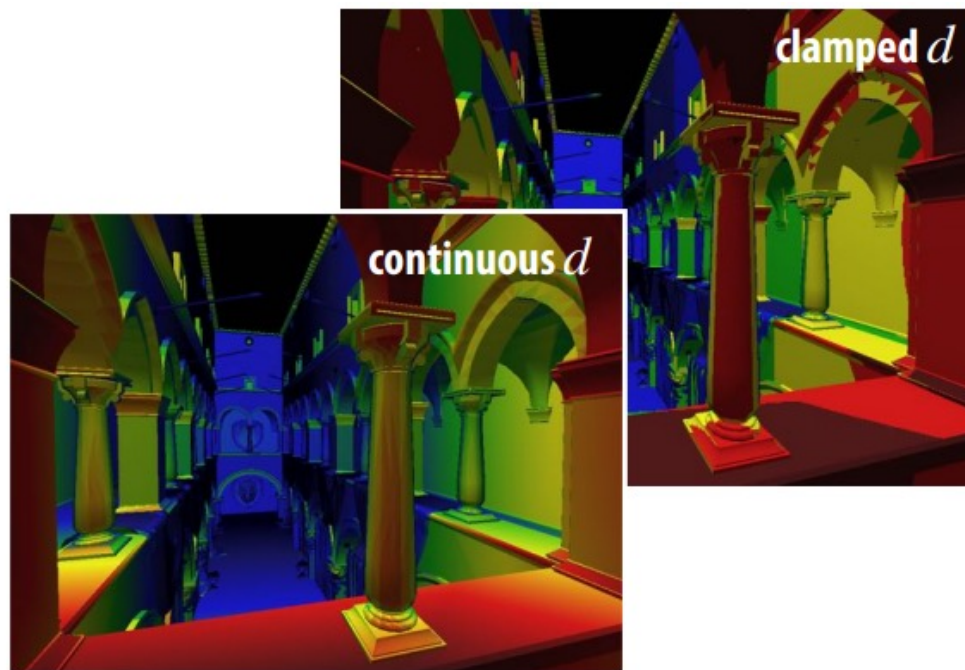
32x32

LOD3

...

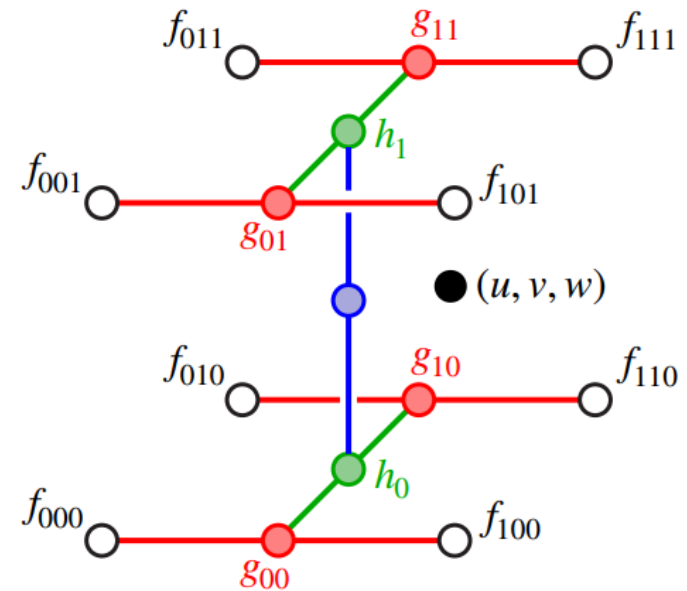
Problem with basic Mipmapping

- If we just use the nearest level, can get artifacts where level “jumps” — appearance sharply transitions from detailed to blurry texture
- Rather than clamping the Mipmap level to the closest integer, interpolating between levels
 - Trilinear Filtering
- Consider perspective distortion
 - Anisotropic Filtering



Trilinear Filtering

- Given a point , and eight closest values
- Just iterate linear filtering:
 - weighted average along
 - weighted average along
 - weighted average along $(u, v, w) \in [0, 1]^3$



OpenGL

```
imgData = LoadAnImage(filename);
unsigned int texture;
glGenTexture(1, &texture);
glBindTexture(GL_TEXTURE_2D, texture);
// set the texture wrapping/filtering options (on the currently bound texture object)
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, w, h, 0, GL_RGB,
GL_UNSIGNED_BYTE, imgData);
glGenerateMipmap(GL_TEXTURE_2D);
```

GLSL

```
//Vertex shader
#version 330
layout (location = 0) in vec3 pos;
layout (location = 1) in vec3 color;
layout (location = 2) in vec2 tex;
out vec3 outColor;
out vec2 texCoord;
void main()
{
    gl_Position = vec4(pos, 1.0); outColor = color; texCoord = tex;
}
```

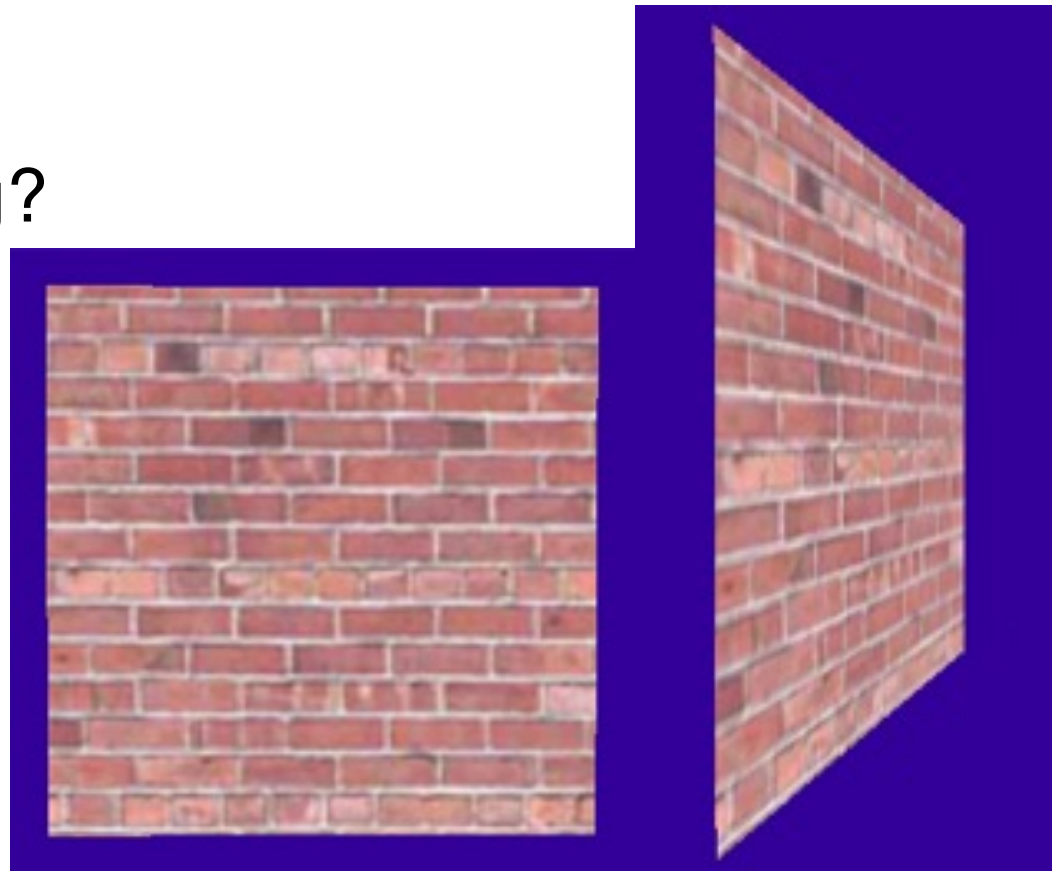
```
//Fragment shader
#version 330
out vec4 FragColor;
in vec3 outColor;
in vec2 texCoord;
uniform sampler2D myTexture;
void main()
{
    FragColor = texture(myTexture, texCoord);
}
```

Other texturing techniques

- Bump mapping
- Displacement mapping
- Environmental mapping

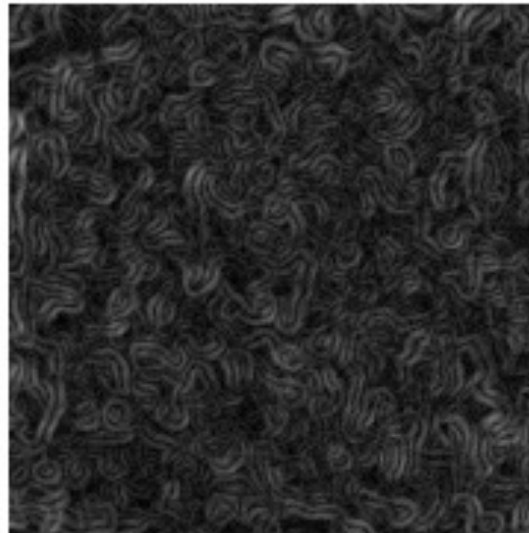
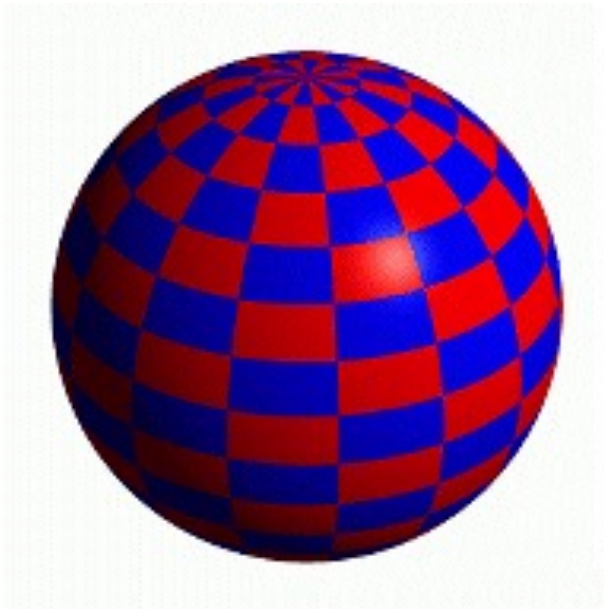
Bump mapping

- Now using textures, we can model a wall using two triangles
- Is it realistic?
- How about shading?

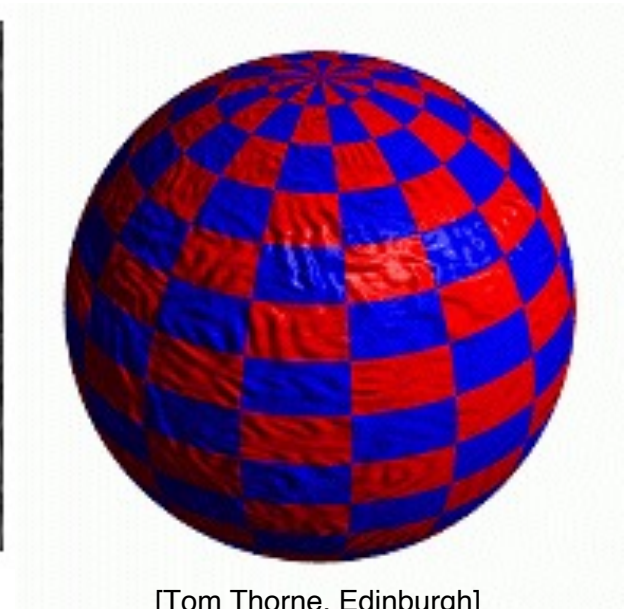


Bump mapping

- Recall that the normal defines the shading and a plane only have one normal for the wall
- We can use texture to disturb the normal!



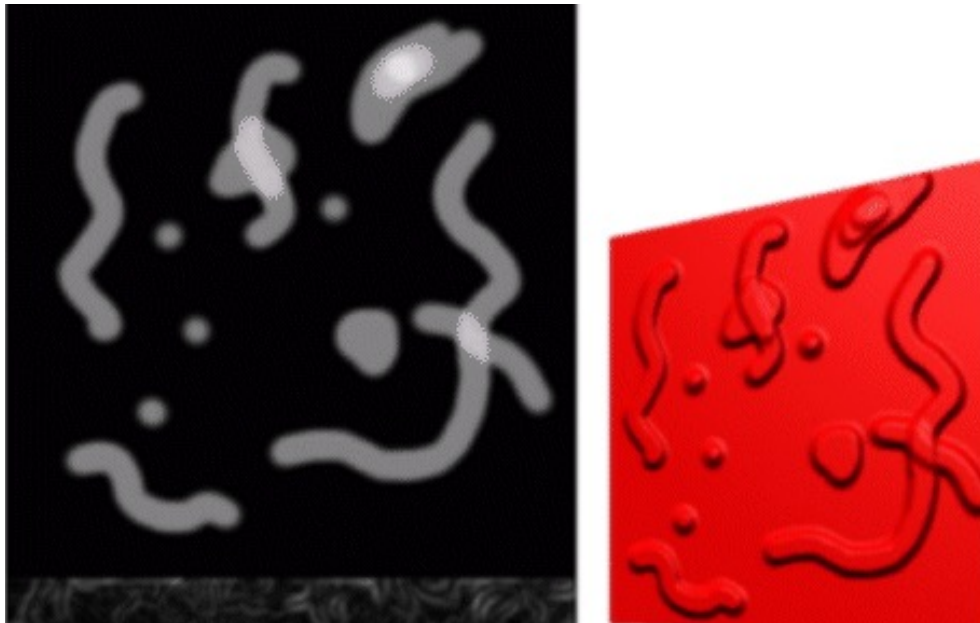
Swirly Bump Map



[Tom Thorne, Edinburgh]

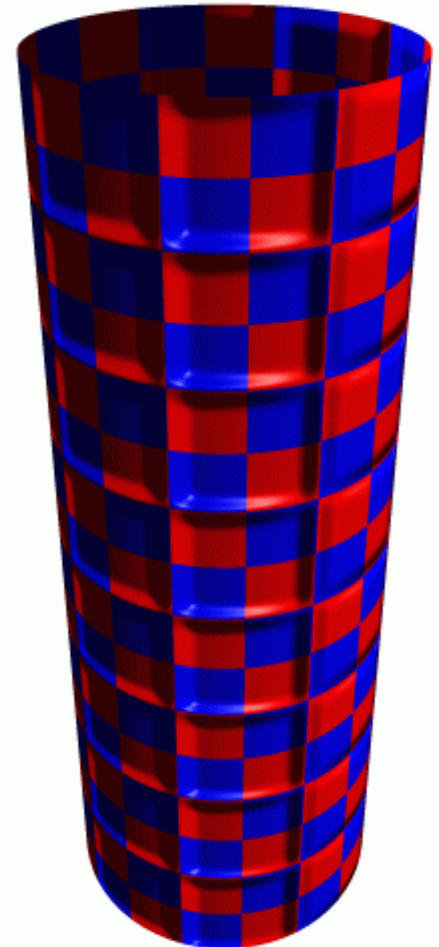
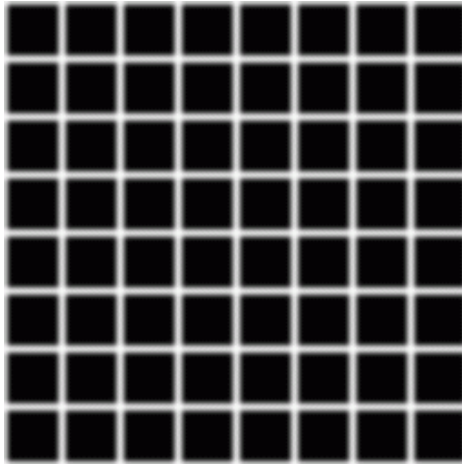
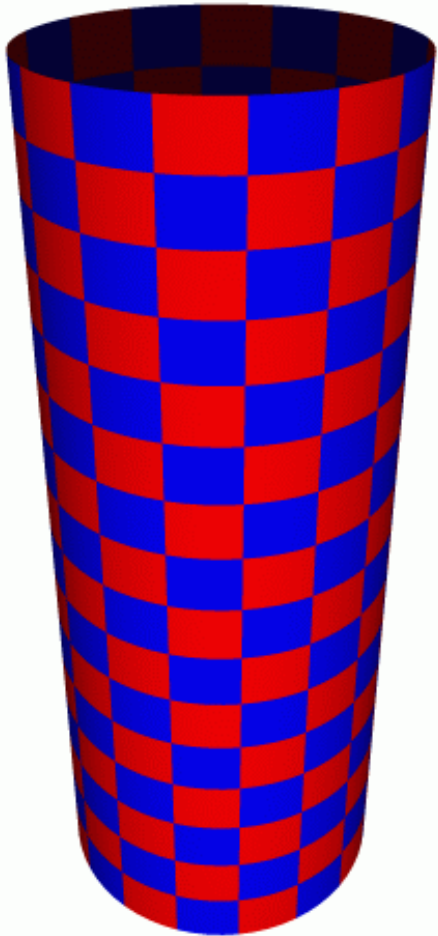
Bump mapping

- Treat the texture as a height function
- Compute the normal from the partial derivatives in the texture



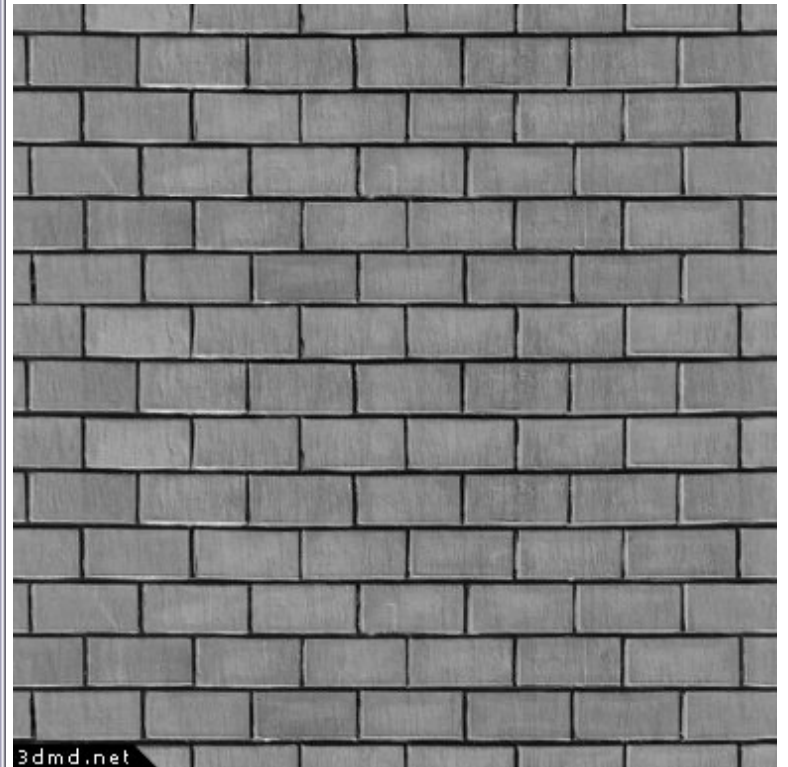
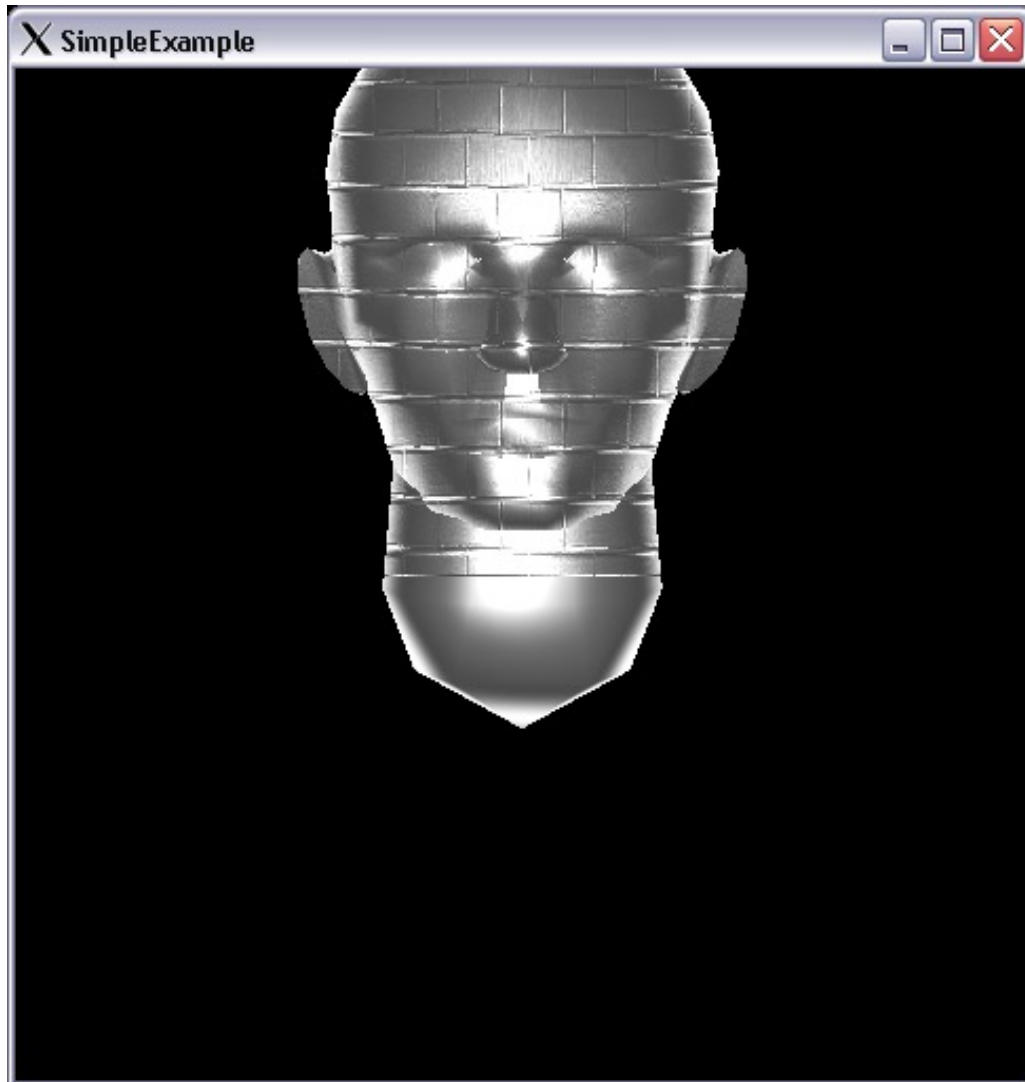
[Tom Thorne, Edinburgh]

Bump Mapping



[Tom Thorne, Edinburgh]

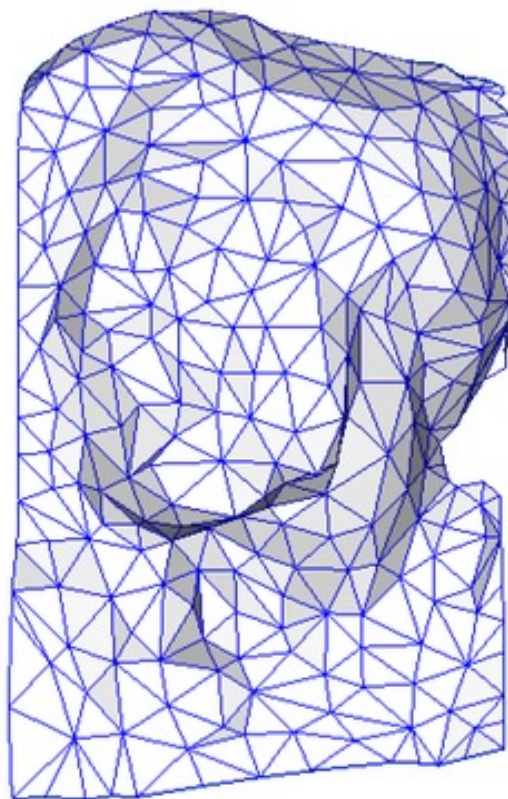
Bump mapping examples



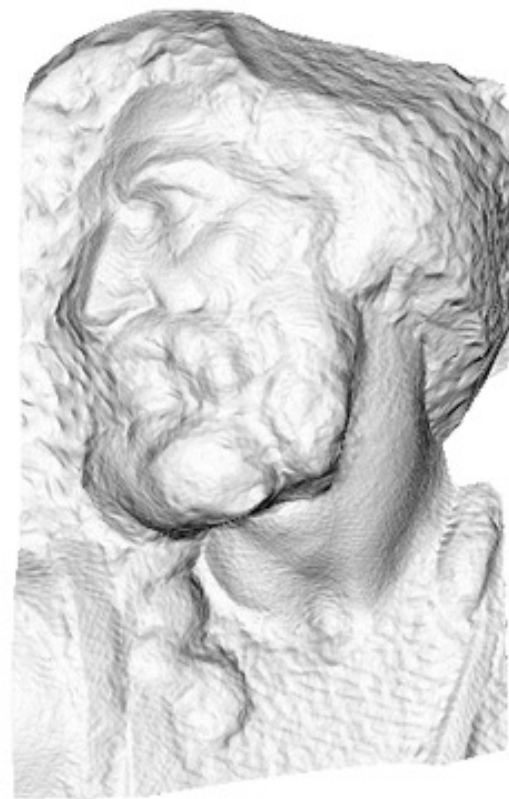
[Tom Thorne, Edinburgh]



original mesh
4M triangles



simplified mesh
500 triangles

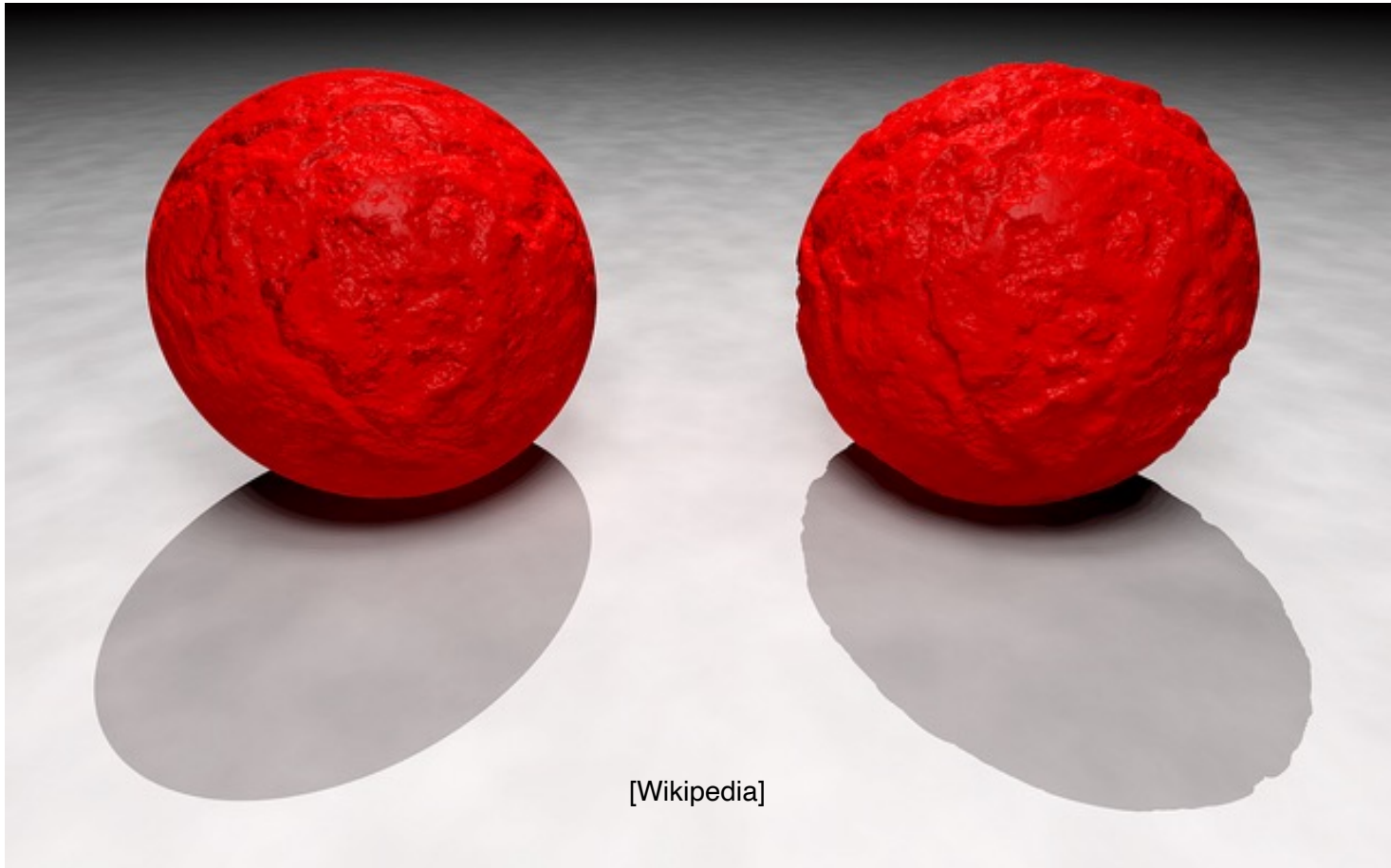


simplified mesh
and normal mapping
500 triangles

[Wikipedia]

Bump mapping examples

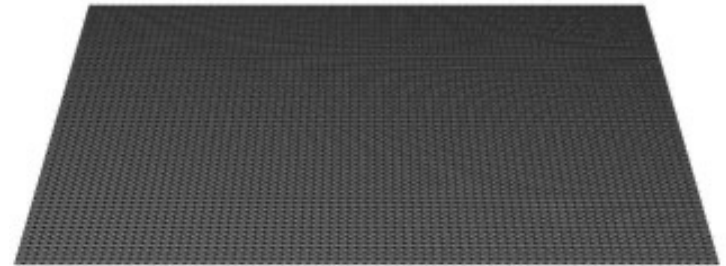
- Notice the difference?



[Wikipedia]

Displacement mapping

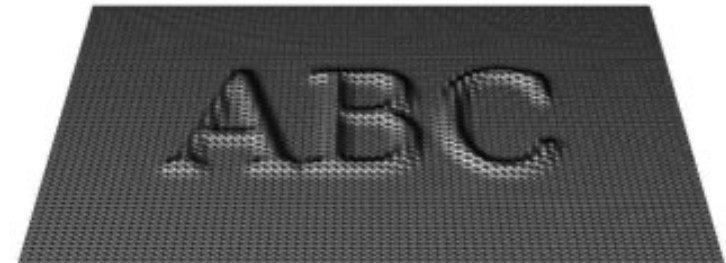
- Use the texture map to actually move the surface point
- The geometry must be displaced before visibility is determined



ORIGINAL MESH

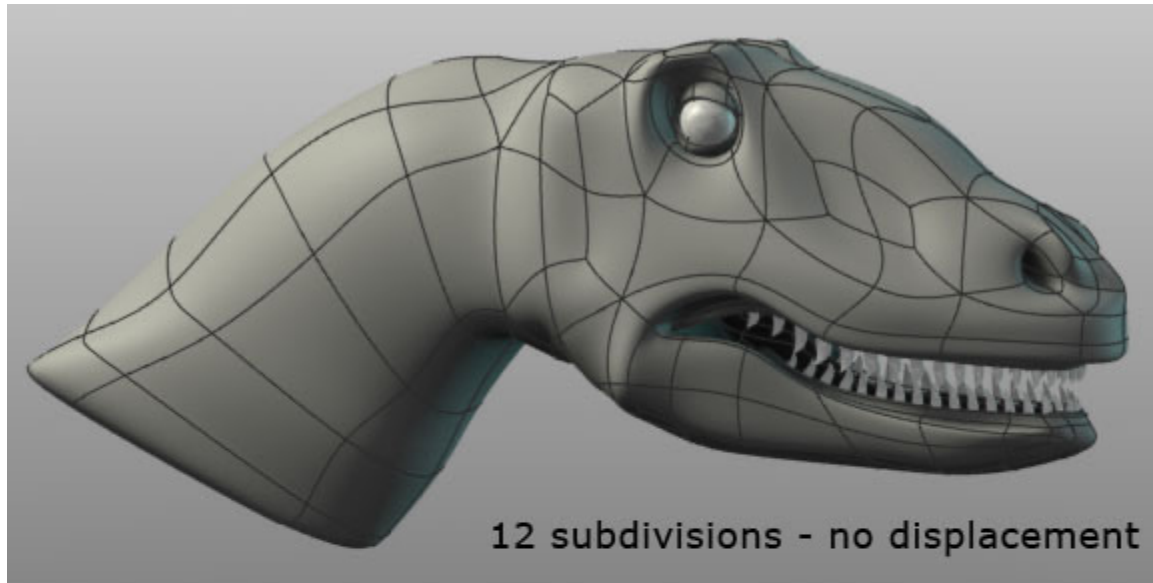


DISPLACEMENT MAP



MESH WITH DISPLACEMENT

[Wikipedia]



12 subdivisions - no displacement



12 subdivisions - displacement

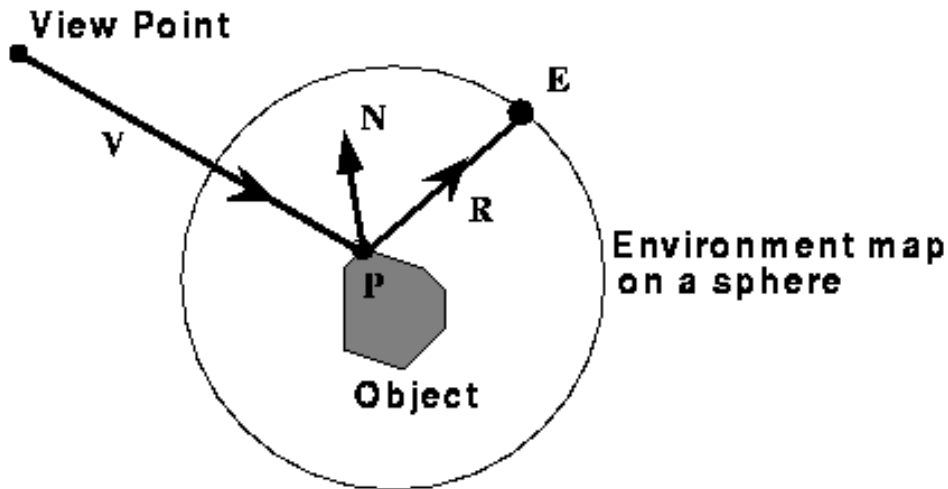
<http://www.outside-hollywood.com/img/profilelg.jpg>



<http://www.cggallery.com/tutorials/displacement/>

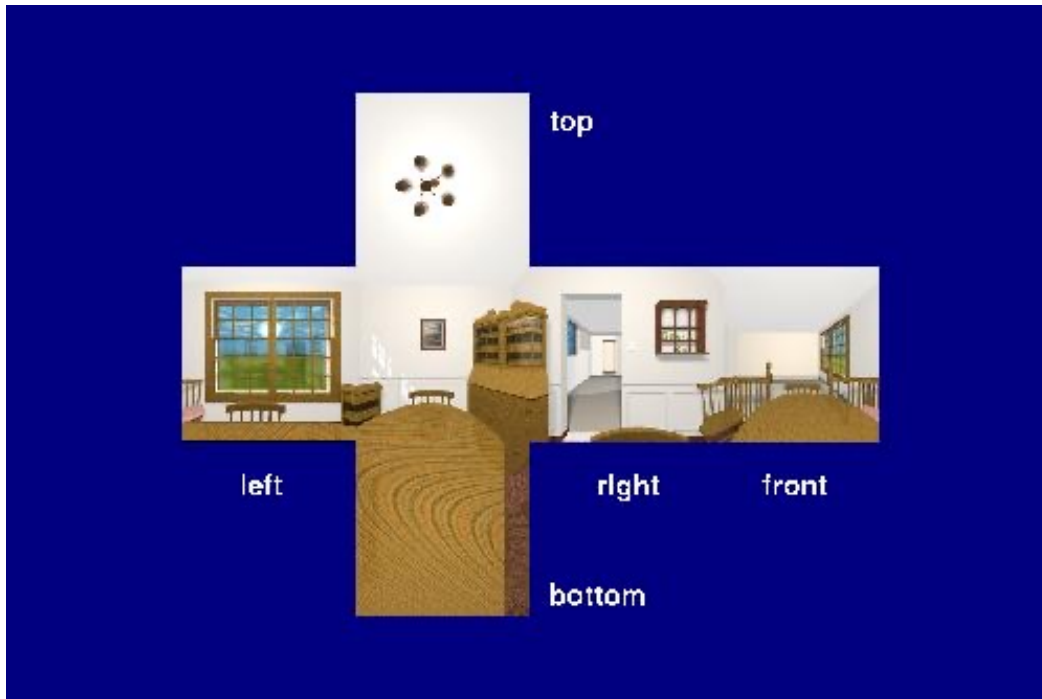
Environment mapping

- It's difficult to generate reflections without GI method
- We can simulate the reflection using texture mapping!

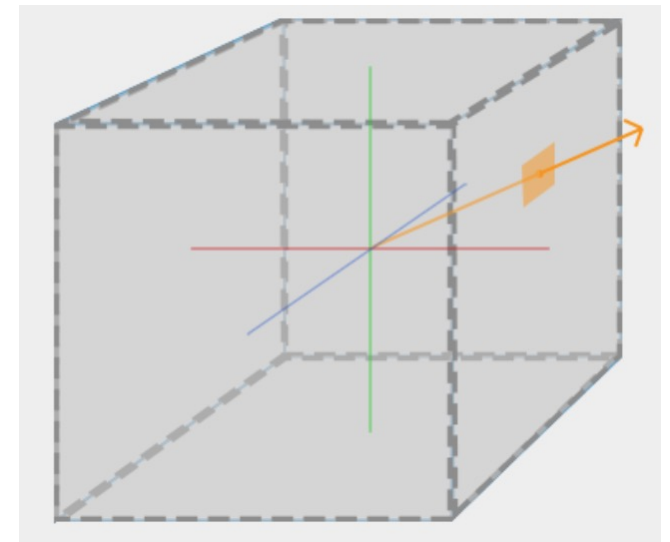


Cubic mapping

- Cubic texture maps
- 2D UV coordinates becomes 3D texture directions



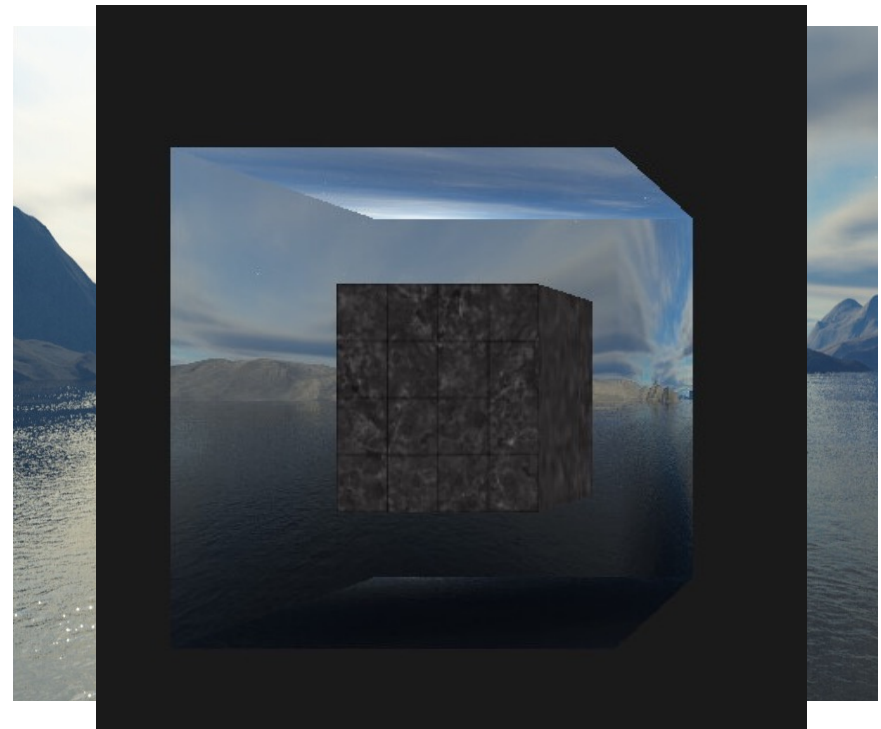
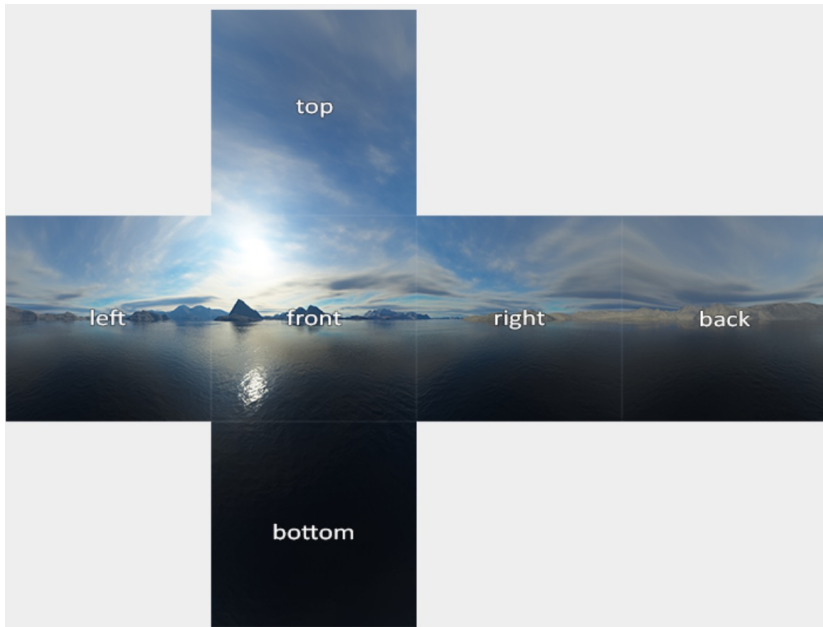
[Rosalee Wolfe / DePaul University]



www.learningopengl.com

The Sky Box

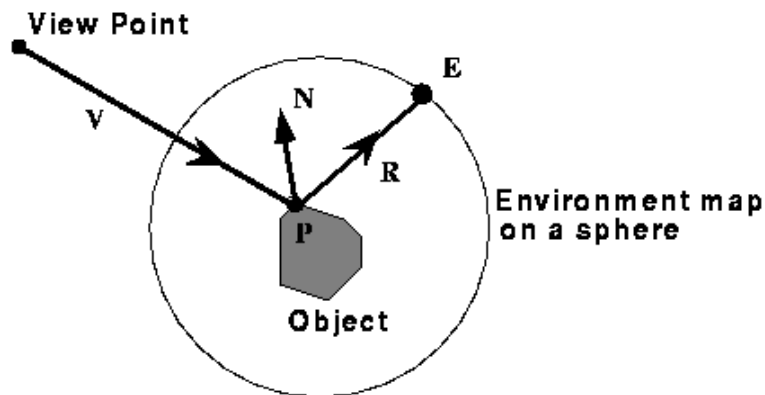
- Render a Box
- Using vertex coordinates as texture directions
- Taking away the translation from view matrix



Environmental mapping

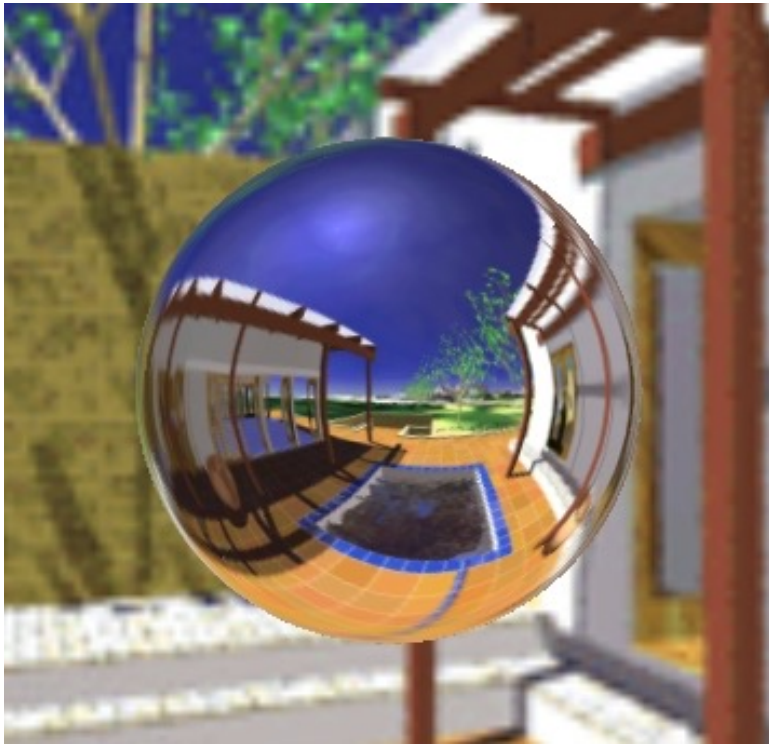
- Calculate proper texture direction in Fragment shader

```
vec3 I = normalize(fragPos - cameraPos);  
vec3 R = reflect(I, normalize(Normal));  
FragColor = vec4(texture(skybox, R).rgb,  
1.0);
```



[Rosalee Wolfe / DePaul University]

Examples



Examples



How to implement refraction?



References

- Ed Angel, CS433 Computer Graphics, NewMexico
- Cutler and Durand, MIT EECS 6.837
- Tom Thorne, COMPUTER GRAPHICS, The University of Edinburgh
- Steve Marschner, CS4620/5620 Computer Graphics, Cornell
- <https://learnopengl.com/Getting-started/Textures>
- <https://learnopengl.com/Advanced-OpenGL/Cubemaps>

- Questions?